Batched Attribute-Based Encryption from Bilinear Pairings

Ramprasad Sarkar[0000−0002−9907−3319], Shayeef Murshid[0009−0008−7042−7911], Mriganka Mandal[0000−0001−8691−4185]

Cryptology and Security Research Unit
R C Bose Centre for Cryptology and Security
Indian Statistical Institute Kolkata, Kolkata, India-700108
{rpsarkar_p,shayeef_r,mriganka}@isical.ac.in

Abstract. Batched identity-based encryption (batched IBE), introduced by Agarwal, Fernando, and Pinkas (CRYPTO 2025), enables a key authority to issue a single succinct key that decrypts an entire batch of ciphertexts. However, existing batched encryption protocols support only identity-based access control and do not accommodate expressive attribute-based policies. We are the first to introduce Batched Attribute-Based Encryption (B-ABE), a generalization of batched encryption to the attribute-based setting. In B-ABE, ciphertexts are associated with attribute sets and batch labels, while a single batch key is bound to an access policy and a public digest of the batch. The key decrypts exactly those ciphertexts in the batch whose attributes satisfy the policy, and its size is independent of the batch size. We construct B-ABE in asymmetric bilinear groups and prove security under the standard Decisional Bilinear Diffie-Hellman (DBDH) assumption in the random oracle model.

We further introduce the first Threshold Batched Attribute-Based Encryption (TB-ABE), in which the master secret is distributed among L authorities via Shamir secret sharing. Any subset of at least T authorities can non-interactively generate publicly verifiable key shares that aggregate into a succinct batch key of the same asymptotic size as in the single-authority setting. We formalize security against static corruptions of up to $(T-1)$ authorities and adaptive key-share queries and prove security under the DBDH assumption in the random oracle model.

Keywords: Attribute-based encryption · Batched key issuance · Threshold cryptography · Encrypted mempools · Bilinear pairings

1 Introduction

Public-key encryption with fine-grained decryption capabilities has been a central theme of modern cryptography. Identity-based encryption (IBE) [8, 26] addresses this problem at the level of identities: anyone may encrypt a message to a public string id, while a trusted authority derives the corresponding decryption key. Attribute-based encryption (ABE) [17, 24] generalizes this paradigm by

replacing identities with expressive access structures. In key-policy ABE, ciphertexts are associated with attribute sets, whereas decryption keys encode access policies, typically represented as linear secret-sharing schemes (LSSS). A ciphertext can be decrypted if and only if its attributes satisfy the policy embedded in the key. Over the past two decades, ABE has become one of the most powerful tools for fine-grained access control, with constructions supporting rich policies, adaptive security, and efficient realizations in bilinear groups [5, 20, 23, 28].

Despite their differences, both IBE [8, 26] and ABE [5, 17, 20, 23, 24, 28] follow the same operational model: a decryption key is issued once and subsequently grants access to all ciphertexts satisfying its authorization condition. In many emerging applications, however, authorization is not intended to be permanent. Instead, it is often tied to a particular event, epoch, block, or execution instance. A regulator may be authorized to inspect only the transactions belonging to a finalized block; an auditor may be granted access only to logs collected during a specific time window; and a committee may release decryption capabilities only after a protocol round has completed. These scenarios suggest that decryption rights should be scoped not merely by identities or policies, but also by a specific collection of ciphertexts.

Motivated by this observation, recent work has introduced the notion of *batched encryption*. The first systematic treatment was given by Agarwal *et al.* [2], who proposed batched identity-based encryption (B-IBE). In their framework, ciphertexts are associated with both an identity id and a batch label tg. Rather than issuing one key per ciphertext, the authority generates a single decryption key that authorizes access to an entire subset of ciphertexts within the batch. Importantly, the size of this key remains independent of both the number of ciphertexts in the batch and the number of authorized identities. The central mechanism enabling this compression is a publicly computable digest that succinctly represents the batch and allows the authority to derive decryption capabilities from the digest alone. Subsequent works have strengthened this paradigm by obtaining standard-model constructions and threshold variants [11, 16, 22, 29].

The appeal of batched encryption is particularly evident in blockchain systems. In encrypted mempool architectures, users encrypt transactions destined for future blocks, preventing adversaries from observing pending transactions before ordering is finalized. Once a block is confirmed, a committee releases a short decryption key that reveals exactly the transactions included in that block. Since the communication required for key release is independent of the number of transactions, batched encryption scales naturally to realistic block sizes. This stands in contrast to earlier threshold-decryption approaches, whose costs grow linearly with the number of ciphertexts.

Yet all existing batched constructions share a common limitation. Decryption remains fundamentally a *membership test*: a ciphertext is decryptable if and only if its identity belongs to a designated set encoded in the batch digest. While sufficient for identity-based access control, this model is too restrictive for many practical scenarios. In many applications, access decisions depend not on identi-

ties but on the semantic properties of the encrypted data. Consider a compliance auditor who wishes to inspect only transactions satisfying a regulatory policy, an incident-response team that needs access only to encrypted logs above a given severity level, or a privacy-preserving auction in which bids should be revealed only when certain eligibility conditions are met. In each case, the natural access condition is expressed as a policy over attributes rather than membership in a predefined set of identities. Consequently, the natural cryptographic abstraction is attribute-based encryption.

At first sight, one might hope that existing techniques already provide this functionality. Unfortunately, neither of the obvious approaches is satisfactory. One could employ a conventional ABE system and issue policy keys directly. While this recovers expressiveness, it loses the notion of batch-scoped authorization entirely: a standard policy key decrypts every present and future ciphertext satisfying the policy. Alternatively, one could attempt to encode a policy as a collection of identities and then apply a batched IBE scheme. Such an approach is inherently impractical, since even moderately expressive policies may correspond to exponentially many satisfying attribute sets. More fundamentally, the algebraic techniques underlying current batched IBE constructions are tailored to membership testing and do not naturally extend to LSSS-based policy evaluation.

These observations reveal a gap between two powerful cryptographic capabilities. Attribute-based encryption offers expressive access control, while batched encryption offers succinct, batch-scoped authorization. Existing techniques achieve one of these goals at the expense of the other. This raises the following question.

> *Can batching be extended from identity membership to expressive attribute-based policies while retaining succinct batch keys?*

Answering this question would significantly broaden the applicability of batched encryption. However, expressiveness alone is not sufficient. Existing batched constructions rely either on generic-group analyses or on non-standard q-type assumptions. Although these assumptions have enabled important progress, they provide a weaker foundation than the standard assumptions that underpin much of pairing-based cryptography. Given that batched encryption is motivated by applications involving financial value, regulatory compliance, and consensus-critical infrastructure, a more conservative foundation is desirable. This motivates a second question.

> *Can batched attribute-based encryption be realized under a standard assumption such as DBDH?*

In the current literature, there is no batched attribute-based encryption. Even if such a similar construction exists, a practical deployment must still address the issue of trust. In all motivating applications, the key-generation authority represents a potential single point of failure. In an encrypted mempool, for example, an authority holding the master secret could decrypt transactions before

their intended release time, undermining the very privacy guarantees the system seeks to provide. The standard cryptographic remedy is to distribute trust among multiple authorities through threshold techniques.

Thresholdizing batched attribute-based encryption is not immediate. A batch key is a structured object containing policy-dependent randomness and secret-sharing information, and independently generated key shares must combine into a valid aggregate key without sacrificing compactness. Furthermore, practical deployments require non-interactive issuance, public verifiability of key shares, and communication costs that remain independent of the batch size. This leads to our final question.

> *Can batched ABE be thresholdized while preserving succinctness, public verifiability, and security?*

1.1 Our Contributions

In this work, we answer all three questions affirmatively. We are the first to introduce the notions of *batched attribute-based encryption* (B-ABE) and *threshold batched attribute-based encryption* (TB-ABE), present concrete constructions in asymmetric bilinear groups, and prove their security under the standard DBDH assumption in the random-oracle model. Our contributions are summarized as follows.

- **Batched Attribute-Based Encryption.** We formalize the notion of (key-policy) batched attribute-based encryption (B-ABE). In this setting, ciphertexts are associated with attribute sets $x \subseteq [N]$ and a batch label tg, while a deterministic and public digest algorithm Digest compresses a batch description $\mathcal{X} = \{x_1, \ldots, x_B\}$ into a digest $d \in \mathbb{Z}_p$. A single execution of BatchKeyGen produces a decryption key bound simultaneously to an LSSS access policy (M, ρ), the batch label tg, and the digest d. We formalize correctness and succinctness, where the key size depends only on the policy dimensions and the sizes of tg and d, and remains independent of the batch size B. We further define an indistinguishability-based security notion that allows adaptive batch-key queries for arbitrary policy, tag, and batch tuples, subject to the natural restriction that challenge-tag queries correspond only to policies not satisfied by any challenge attribute set.
- **Construction and Security of B-ABE.** We present a concrete B-ABE construction in asymmetric bilinear groups. For an LSSS matrix with ℓ_0 rows, the resulting batch key consists of $3\ell_0$ elements of $\mathbb{G}_2$, independent of the batch size B, while each ciphertext contains only $|x| + 3$ group elements, where x is the attribute set. A digest-binding mechanism incorporates the value $d = H(\mathsf{tg}, B, x_1, \ldots, x_B, [\alpha]_T)$ into the issued key and revalidates it during decryption. This ensures that keys generated under the same tag but for different batches cannot be substituted for one another, providing a hash-based analogue of the algebraic binding achieved through KZG commitments in batched IBE [2]. We prove selective security under the standard

DBDH assumption in the random-oracle model. Furthermore, the construction can be extended to achieve adaptive security through complexity leveraging [7]. In contrast, applying the dual-system encryption methodology [27] would require a substantial redesign of the scheme to accommodate semi-functional keys and ciphertexts. Therefore, we adopt complexity leveraging as a construction-preserving alternative.

- **Threshold Batched Attribute-Based Encryption.** We introduce the notion of threshold batched attribute-based encryption (TB-ABE), involving a dealer, L decryption authorities, and a threshold parameter $T \leq L$. In our construction, the master secret exponent α is distributed using (T, L)-Shamir secret sharing. The dealer publishes Feldman commitments $\{[a_j]_1\}_{j=0}^{T-1}$ to the sharing polynomial, enabling each authority to verify its share locally using $O(T)$ exponentiations and without interaction. Each authority independently computes a partial batch key consisting of $3\ell_0$ elements of $\mathbb{G}_2$ together with $k-1$ auxiliary commitments in $\mathbb{G}_1$, where the communication and computation costs are independent of both the batch size B and the number of authorities L. A deterministic pairing-based verification algorithm, VerifyKeyShare, makes every partial key publicly verifiable and provides robustness against malformed or inconsistent shares. Public aggregation is achieved through standard Lagrange interpolation and requires no secret information.
- **Security of TB-ABE.** We formalize and analyze static security for TB-ABE protocol. In this model, an adversary may initially corrupt any subset $C \subseteq [L]$ with $|C| < T$, obtaining the corresponding secret shares, and may subsequently issue adaptive key-share queries to honest authorities throughout the attack. Such queries may include challenge-tag requests, provided that the associated policies do not satisfy the challenge attribute sets. We prove security under the DBDH assumption in the random-oracle model. The proof embeds the DBDH challenge into the Shamir sharing polynomial through an interpolation polynomial f satisfying $f(0) = 1$ and vanishing on all corrupted indices. Consequently, corrupted shares remain identically distributed to those in the real system, while the embedded hardness instance is carried exclusively by honest shares. In addition, we establish correctness, completeness, and succinctness of the threshold construction unconditionally.

1.2 Related Work

Attribute-based encryption. Attribute-based encryption (ABE) originates in the fuzzy IBE framework of Sahai *et al.* [24] and was subsequently formalized in its key-policy and ciphertext-policy variants by Goyal *et al.* [17] and Bethencourt *et al.* [5], respectively. Subsequent works significantly expanded the expressiveness and security of ABE. Waters [28] obtained expressive constructions supporting general LSSS access structures under static assumptions, while Lewko *et al.* [20] and Lewko *et al.* [19] introduced dual-system techniques that

enabled adaptive security. Rouselakis *et al.* [23] later removed restrictions on the attribute universe. Despite these advances, existing ABE systems follow the traditional authorization model: a policy key remains valid for all ciphertexts satisfying the policy. In particular, they provide neither batch-scoped authorization nor a mechanism for binding decryption capabilities to a publicly specified batch of ciphertexts.

Batched IBE and encrypted mempools. A recent line of work has investigated succinct decryption capabilities for collections of ciphertexts. Agarwal *et al.* [2] introduced batched IBE, together with a threshold variant, using KZG commitments [18] and modified BLS techniques [10]. Their construction is proven secure in the generic group model and achieves key issuance costs independent of the batch size. Gong *et al.* [16] subsequently developed an algebraic framework for batched IBE in the plain model, obtaining selectively secure constructions from q-type assumptions in prime-order groups and adaptively secure constructions in composite-order groups, together with threshold extensions. Boneh *et al.* [9] further studied threshold batched IBE without epoch-based key release. Closely related systems include Ferveo [3], which employs per-ciphertext threshold decryption for encrypted mempools, and BTX [1], which explores batch threshold encryption. All of these approaches, however, remain identity-based: decryption is determined by identity membership and does not support expressive attribute-based access policies.

Threshold cryptography and Multi-authority ABE. Threshold cryptography dates back to the seminal work of Desmedt *et al.* [14]. Feldman [15] introduced publicly verifiable secret sharing, which underlies our dealer-verification mechanism, while Boldyreva [6] demonstrated non-interactive threshold issuance for pairing-based signatures. A related direction is multi-authority and decentralized ABE [21], where control over attributes is distributed among multiple authorities. These systems address a different problem from ours: they decentralize attribute issuance rather than the generation of a single batched decryption key. Consequently, they do not provide batch-size-independent key shares, publicly verifiable policy-key shares, or a batched decryption interface.

2 Technical Overview

This section presents a high-level technical overview of our constructions. We proceed in four steps. We first explain the design of our Batched Attribute-Based Encryption (B-ABE) scheme, in which a single succinct secret key decrypts an entire batch of ciphertexts while remaining cryptographically bound to that exact batch (Section 2.1). We then sketch the security proof, which combines a tag-level partitioning strategy with a hybrid argument over the batch under the Decisional Bilinear Diffie–Hellman (DBDH) assumption. Next, we show how the algebraic linearity of our key structure allows us to distribute the key-generation algorithm among L authorities via Shamir secret sharing, yielding a Threshold Batched ABE (TB-ABE) scheme with publicly verifiable key shares. Finally, we

discuss how the single-authority security proof is upgraded to tolerate static corruptions of up to $(T-1)$ authorities.

Notations. We denote the security parameter by $\lambda \in \mathbb{N}$ and assume that every algorithm receives it in unary as 1^λ. For $n \in \mathbb{N}$, we write $[n] = \{1, \ldots, n\}$. An algorithm is PPT if it runs in probabilistic polynomial time in λ. For a finite set S, $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from S; for a (possibly randomised) algorithm $\mathcal{A}$ we write $y \leftarrow \mathcal{A}(\cdots)$ for its output, $|S|$ for the cardinality of S, and $\perp$ for a distinguished failure symbol. Moreover, column vectors in lower-case boldface (e.g. $\boldsymbol{y}$), and matrices in upper-case boldface (e.g. $\mathbf{M}$); $(\cdot)^\top$ denotes transpose, $\mathbf{M}_i$ the i-th row of $\mathbf{M}$, $\mathbf{M}_{i,j}$ its (i,j)-th entry, and $\mathbf{e}_1$ the first standard basis vector. We write $\mathbb{Z}_p$ for the integers modulo a prime p and $\mathbb{Z}_p^k$ for the corresponding vector space. For the asymmetric (Type-3) bilinear groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order p introduced below, we adopt the implicit representation $[x]_1 = g_1^x$, $[x]_2 = g_2^x$, and $[x]_T = e(g_1, g_2)^x$ for $x \in \mathbb{Z}_p$; group operations are written multiplicatively, so that $[x]_1 \cdot [x']_1 = [x + x']_1$ and $e([x]_1, [y]_2) = [xy]_T$, and this notation extends component-wise to vectors and matrices over $\mathbb{Z}_p$. Finally, $\mathcal{M}$, $\mathcal{T}$, and $[N]$ denote the message space, the tag (batch-label) space, and the attribute universe, respectively, while $\mathcal{X} = \{x_1, \ldots, x_B\}$ denotes a size-B batch of attribute sets with each $x_b \subseteq [N]$.

2.1 Batched ABE: One Succinct Key for an Entire Batch

Starting Point. In a key-policy attribute-based encryption scheme, a ciphertext is associated with an attribute set $x \subseteq [N]$, while a secret key is generated for an access policy $(\mathbf{M}, \rho)$. Decryption is possible whenever the attribute set x satisfies the policy encoded by $(\mathbf{M}, \rho)$. We are interested in a setting in which ciphertexts are produced and consumed in large batches. Let tg denote a public batch identifier. During an epoch associated with tg, a sender generates a collection of ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_B$, under attribute sets $x_1, \ldots, x_B$, respectively. A recipient possessing an access policy $(\mathbf{M}, \rho)$ wishes to decrypt precisely those ciphertexts within the batch whose attributes satisfy the policy.

Standard ABE does not provide an adequate mechanism for granting such batch-specific access. On the one hand, the authority may generate a separate decryption key for each ciphertext in the batch. While this achieves the desired restriction to a particular collection of ciphertexts, the size of the key scales linearly with the batch size B. On the other hand, the authority may issue a single conventional policy key. Such a key is compact, but it authorizes decryption of all ciphertexts satisfying the policy, including ciphertexts generated in future epochs. Consequently, the authority loses the ability to confine decryption rights to a specific batch that has been approved or audited. Our objective is to obtain a primitive that combines the advantages of both approaches. Specifically, we seek:

- **Succinctness**: the authority should issue a single batch key whose size is independent of the number of ciphertexts B; and

– **Batch binding**: the issued key should be valid only for the designated batch $(\mathsf{tg}, x_1, \ldots, x_B)$, and should not enable decryption of any other collection of ciphertexts, including ciphertexts associated with the same tag tg.

To capture these requirements, we introduce a new primitive called Batched Attribute-Based Encryption (B-ABE). A B-ABE scheme consists of the algorithms (Setup, BatchKeyGen, Enc, Digest, BatchDec). At a high level, B-ABE enables the authority to derive a compact batch-specific decryption key that authorizes access precisely to the ciphertexts belonging to a designated batch while preserving the expressive policy functionality of attribute-based encryption.

Core Idea (Tag-Bound Ciphertexts). Our construction builds upon a pairing-based KP-ABE key structure. The master secret key is defined as $\mathsf{msk} = (\alpha, v, h, \{u_i\}_{i\in[N]})$, while the master public key publishes the group elements $[v]_1$, $[h]_1$, $\{[u_i]_1\}_{i\in[N]}$, together with $[\alpha]_T$. The central design principle is to make both ciphertexts and secret keys tag-aware, thereby cryptographically binding them to a designated tag.

To encrypt a message under an attribute set x and tag tg, the encryptor first computes $\tau \leftarrow H(\mathsf{tg})$ and samples a fresh exponent $s \leftarrow \mathbb{Z}_p$. The ciphertext is then formed as

$$C_0 = m \cdot [\alpha s]_T, \qquad C_1 = [s]_1, \qquad C_2 = [(v + h\tau)s]_1, \qquad C_j = [u_j s]_1 \ \text{ for } j \in x.$$

The component C_2 serves as a tag anchor. Specifically, the pair (v, h) links the ciphertext to the tag via their hash values. The same tag-dependent quantity is incorporated into every row of a decryption key. Given an access policy represented by an LSSS matrix $(\boldsymbol{M}, \rho)$, the algorithm BatchKeyGen secret-shares α to obtain shares $\{\lambda_i\}_{i\in[\ell]}$ and samples fresh randomizers $r_i, t_i \leftarrow \mathbb{Z}_p$ for each row. It outputs

$$K_{i,0} = [\lambda_i + (v + h\tau)r_i - u_{\rho(i)}t_i]_2, \qquad K_{i,1} = [r_i]_2, \qquad K_{i,2} = [t_i]_2.$$

During decryption, computer $D_{b,i} = e(C_{b,1}, K_{i,0}) \cdot e(C_{b,2}, K_{i,1})^{-1} \cdot e(C_{b,\rho(i)}, K_{i,2})$. A straightforward expansion of the exponents shows that the masking terms $(v + h\tau)r_i$ and $u_{\rho(i)}t_i$ cancel precisely when the ciphertext and key are associated with the same tag. In this case, one obtains $D_{b,i} = [s_b\lambda_i]_T$. Using the reconstruction coefficients ω_i of the underlying LSSS, the decryptor recovers $\prod_i D_{b,i}^{\omega_i} = [\alpha s_b]_T$ which enables recovery of the message m_b.

The tag-binding property follows directly from this cancellation mechanism. If a key generated for tag tg is applied to a ciphertext carrying a different tag tg', then $\tau \neq \tau'$ and the cancellation no longer holds. Instead, an additional term $h(\tau - \tau')r_i s$ remains in the exponent, preventing successful decryption. Consequently, a key is algebraically bound to its designated tag and provides no useful information about ciphertexts associated with any other tag. An important feature of the construction is that the same collection of key components $\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i\in[\ell]}$ is reused across all B ciphertexts in a batch. Each ciphertext contributes its own fresh randomness s_b, whereas the key contains only

the policy-dependent shares and randomizers. As a result, the key size remains fixed at $3\ell_0$ group elements, independent of the batch size B. This immediately yields a succinct key representation while supporting decryption of an arbitrary number of ciphertexts associated with the same tag.

Binding a Decryption Key to a Specific Batch via a Digest. Tag-awareness alone is insufficient to ensure batch-specific binding. In particular, any two batches generated under the same tag tg would remain mutually interchangeable. To eliminate this ambiguity, we incorporate a second, lightweight binding mechanism based on a batch digest. Specifically, the deterministic algorithm Digest compresses the complete batch description into a single field element,

$$d \;=\; H\big(\underbrace{\mathsf{tg},\, B,\, x_1, \ldots, x_B,\, [\alpha]_T}_{\mathsf{desc}}\big) \;\in\; \mathbb{Z}_p,$$

which serves as a compact commitment to the batch. The value d is supplied as input to $\mathsf{BatchKeyGen}$ and embedded into the resulting decryption key. During decryption, $\mathsf{BatchDec}$ first reconstructs the digest $d' = \mathsf{Digest}(\mathcal{X}, \mathsf{tg}, \mathsf{mpk})$ from the ciphertexts presented for decryption. The algorithm proceeds only if $d' = d$; otherwise, it aborts before performing any pairing computations. This verification step guarantees that the key is used exclusively with the batch for which it was issued. Because the digest incorporates the tag, the batch cardinality, and every attribute set contained in the batch, any two keys generated under the same tag but for different batches necessarily carry distinct digest values. Consequently, they cannot be substituted for one another without being detected. Under the collision-resistance of H in the random-oracle model, an adversary can produce a different batch yielding the same digest only with negligible probability.

2.2 Proving Security: Partitioning at the Tag Level

Proof Strategy. We establish (selective) security under the standard DBDH assumption. Recall that the assumption states that given $(g_1, g_2, [a]_1, [a]_2, [b]_1, [b]_2, [c]_1)$, it is computationally infeasible to distinguish $[abc]_T$ from a uniformly random element of $\mathbb{G}_T$. In the selective-security game, the adversary first commits to a challenge tag tg^* together with challenge attribute sets $x_1^*, \ldots, x_B^*$. It may subsequently request batch decryption keys for arbitrary tuples $(\boldsymbol{M}, \rho, \mathsf{tg}, \mathcal{X})$, subject to the standard admissibility requirement: whenever $\mathsf{tg} = \mathsf{tg}^*$, the queried policy must not authorize any challenge attribute set. Finally, the adversary receives encryptions of one of two challenge message vectors and must determine the encrypted vector. The proof proceeds via a sequence of hybrid games $\mathsf{Game}_0^{(\beta)}, \mathsf{Game}_1^{(\beta)}, \ldots, \mathsf{Game}_B^{(\beta)}$, where, in $\mathsf{Game}_j^{(\beta)}$, the masking component $C_{b,0}$ of the first j challenge ciphertexts is replaced by an independent uniform element of $\mathbb{G}_T$. In the terminal hybrid, all B challenge masking components are uniform, rendering the challenge ciphertexts statistically independent of the challenge bit. Consequently, the adversary's advantage vanishes in the final game. Each tran-

sition between adjacent hybrids is reduced to a single DBDH challenge, yielding an overall loss proportional to the number of hybrids.

Challenge Embedding via Tag-Based Partitioning. The central technical ingredient is a partitioning argument carried out at the level of tags rather than attributes. The reduction programs the random oracle so that $H(\mathsf{tg}^*) = \tau^*$ for a known value τ^* and implicitly defines

$$\alpha = ab + \tilde{\alpha}, \qquad v = -b\tau^* + \tilde{v}, \qquad h = b + \tilde{h}, \qquad u_i = \tilde{u}_i,$$

where the tilded quantities are chosen uniformly by the simulator. This programming induces two complementary effects. First, at the challenge tag tg^*, the dependence on the unknown value b disappears from the tag anchor $v + h\tau^* = \tilde{v} + \tilde{h}\tau^*$. As a result, the simulator can generate the challenge ciphertext using only the element $[c]_1$, implicitly setting the encryption randomness to $s_j = c$. For the hybrid index j, it computes $C_{j,1} = [c]_1$, $C_{j,2} = [c]_1^{\tilde{v}+\tilde{h}\tau^*}$, $C_{j,k} = [c]_1^{\tilde{u}_k}$, and embeds the DBDH challenge element $\mathcal{T}$ into $C_{j,0} = m_\beta^{(j)} \cdot \mathcal{T} \cdot e([c]_1, g_2)^{\tilde{\alpha}}$. If $\mathcal{T} = [abc]_T$, then $C_{j,0} = m_\beta^{(j)} \cdot e(g_1, g_2)^{abc} \cdot e(g_1, g_2)^{\tilde{\alpha}c} = m_\beta^{(j)} \cdot e(g_1, g_2)^{\alpha c}$, which is distributed exactly as in a real encryption. On the other hand, if T is uniform in $\mathbb{G}_T$, then $C_{j,0}$ is itself uniform and independent of the encrypted message. This precisely captures the difference between adjacent hybrids.

Second, for any queried tag $\mathsf{tg} \neq \mathsf{tg}^*$, letting $\tau = H(\mathsf{tg})$ and $\Delta = \tau - \tau^* \neq 0$, the dependence on b survives: $v + h\tau = (\tilde{v} + \tilde{h}\tau) + b\Delta$. This residual term is exactly what enables simulation of key queries. The only unknown in the key exponent is the term $M_{i,1}\, ab$ contributed by λ_i. The reduction implicitly re-randomizes

$$r_i \;=\; \tilde{r}_i - \frac{M_{i,1}\, a}{\Delta}, \qquad t_i = \tilde{t}_i,$$

so that the cross term $(v+h\tau)\, r_i$ contributes $b\Delta \cdot (-M_{i,1}a/\Delta) = -M_{i,1}ab$, exactly cancelling the unknown ab part of λ_i. So, all of $K_{i,0}, K_{i,1}, K_{i,2}$ are computable from $[a]_2, [b]_2$; and since $\tilde{r}_i$ is uniform, the simulated key is distributed identically to a real one.

2.3 Lifting to the Threshold Setting

Although B-ABE achieves succinct batch decryption, it still relies on a single trusted key-generation authority that holds the master secret α. Consequently, compromise of this authority immediately compromises every batch key in the system. To eliminate this single point of trust, we distribute key-generation functionality among L decryption authorities such that any subset of at least T authorities can jointly generate a valid batch key, whereas any coalition of fewer than T authorities learns no information about the underlying secret.

The design is guided by three requirements. First, authorities should operate non-interactively; each authority must be able to compute its contribution independently from its local secret share. Second, generated key shares should be publicly verifiable, allowing a decryptor to identify and discard malformed

contributions before aggregation. Third, the compactness properties of B-ABE must be retained: both individual key shares and the final aggregated key should remain independent of the batch size B, while the aggregated key should additionally remain independent of the number of participating authorities.

Leveraging Linearity of the Key Structure. The threshold construction follows from a simple but crucial observation: the entire batch-key generation procedure is linear in both the master secret α and the randomness used during key generation. In particular, the exponent of $K_{i,0}$ takes the form $\lambda_i + (v + h\tau)r_i - u_{\rho(i)}t_i$, where $\lambda_i = \boldsymbol{M}_i\boldsymbol{y}$ is linear in the vector $\boldsymbol{y} = (\alpha, y_2, \ldots, y_k)^\top$. Moreover, the public parameters v, h, and $u_{\rho(i)}$ are common to all authorities. This linear structure naturally composes with Shamir secret sharing in the exponent. A trusted dealer first generates the master secret key as in the single-authority scheme and then constructs a (T, L)-Shamir sharing of α by sampling a polynomial f_α and defining $\alpha_\ell = f_\alpha(\ell)$. Authority ℓ receives the secret share $\mathsf{sk}_\ell = (\alpha_\ell, v, h, \{u_j\}_{j\in[N]})$. To contribute to a batch key, authority ℓ runs CompKeyShare, which is *exactly* the single-authority BatchKeyGen with α_ℓ in place of α: it samples its own vector $\boldsymbol{y}^{(\ell)} = (\alpha_\ell, y_2^{(\ell)}, \ldots, y_k^{(\ell)})^\top$ and randomness $r_i^{(\ell)}, t_i^{(\ell)}$, and outputs rows $(K_{i,0}^{(\ell)}, K_{i,1}^{(\ell)}, K_{i,2}^{(\ell)})$. Given T shares from a committee U, the aggregation algorithm CompKeyAgg simply takes the Lagrange combination row-wise in the exponent:

$$\bar{K}_{i,b} \;=\; \prod_{\ell\in U} \big(K_{i,b}^{(\ell)}\big)^{\mu_\ell^U}, \qquad b \in \{0,1,2\}.$$

Because $v, h, u_{\rho(i)}$ factor out of the interpolation, the aggregate is a perfectly formed batch key for the implicit values

$$\bar{\lambda}_i = \textstyle\sum_\ell \mu_\ell^U \lambda_i^{(\ell)}, \qquad \bar{r}_i = \sum_\ell \mu_\ell^U r_i^{(\ell)}, \qquad \bar{t}_i = \sum_\ell \mu_\ell^U t_i^{(\ell)},$$

whose LSSS reconstruction satisfies $\sum_i \omega_i \bar{\lambda}_i = \sum_{\ell\in U} \mu_\ell^U \, \alpha_\ell = \alpha$.

Consequently, the aggregated key is functionally equivalent to a key generated by the original B-ABE scheme. Batch decryption proceeds unchanged, including verification of the digest condition $d' \stackrel{?}{=} d$. The aggregated key contains only $3\ell_0$ elements of $\mathbb{G}_2$ together with the digest value, and is therefore independent of both the batch size B and the number of participating authorities $|U|$. Hence, the succinctness guarantees of B-ABE are fully preserved.

Public Verifiability. To ensure robustness against malicious or faulty participants, we incorporate two complementary non-interactive verification mechanisms. Both are based on lightweight commitments that can be published alongside existing protocol values.

- Following Feldman VSS, the dealer publishes commitments $\{[a_j]_1\}_{j=0}^{T-1}$ to the coefficients of f_α. Authority ℓ verifies its share by checking $[\alpha_\ell]_1^* = \prod_{j=0}^{T-1} [a_j]_1^{\ell^j}$ thereby confirming that $\alpha\ell$ is a valid evaluation of the committed polynomial.

– Each authority's public key $\mathsf{pk}_\ell = [\alpha_\ell]_T$ is published, and every key share σ_ℓ additionally carries commitments $\{[y_j^{(\ell)}]_1\}_{j=2}^{k}$ to the non-secret coordinates of its LSSS vector. Any party can then reconstruct a commitment to each LSSS share in the target group, $[\lambda_i^*]_T = \mathsf{pk}_\ell^{M_{i,1}} \cdot \prod_{j\geq 2} e([y_j^{(\ell)}]_1, g_2)^{M_{i,j}}$, and verify each row of σ_ℓ by a single pairing-product equation,

$$e(g_1, K_{i,0}^{(\ell)}) \stackrel{?}{=} e\big([v]_1 + \tau[h]_1,\, K_{i,1}^{(\ell)}\big) \cdot e\big([u_{\rho(i)}]_1,\, K_{i,2}^{(\ell)}\big)^{-1} \cdot [\lambda_i^*]_T,$$

using only mpk.

Any share satisfying these checks is guaranteed to be consistent with the committed secret share $\alpha\ell$. Consequently, any set of T verified shares aggregates into a valid decryption key.

2.4 Static Security for TB-ABE

Threat Model. We consider a static security model for TB-ABE protocol. Before setup, the adversary selects a corruption set $C \subset [L]$ with $|C| < T$ and obtains the corresponding secret shares $\{\mathsf{sk}_\ell\}_{\ell\in C}$. It may then adaptively query key shares for arbitrary policies, batches, and tags, subject to the conditions that (i) every challenge attribute set is unauthorized for any policy queried at the challenge tag tg^*, and (ii) each honest authority is queried at most once on tg^*. The latter restriction is necessary since shares issued for the same tag can be linearly combined, making repeated queries equivalent to rerunning key generation.

Simulating Shares around the DBDH. The reduction extends the single-authority simulation by ensuring that the embedded DBDH secret resides entirely in the honest shares. It maintains the parameterization $\alpha = ab+\tilde{\alpha}$, $v = -b\tau^* + \tilde{v}$, $h = b + \tilde{h}$ while generating corrupted shares honestly. Let f be a $(T-1)$-degree polynomial satisfying $f(0) = 1$ and $f(\ell) = 0$ for all $\ell \in C$. For each honest authority, $\alpha_\ell = \tilde{\alpha}_\ell + f(\ell)\cdot ab, \quad \ell \in [L] \setminus C$. This yields a valid sharing of α, remains consistent with the revealed corrupted shares, and allows computation of the public parameters via $\mathsf{pk}_\ell = [\tilde{\alpha}_\ell]_T \cdot e([a]_1, [b]_2)^{f(\ell)}$. For key-share queries at tags $\mathsf{tg} \neq \mathsf{tg}^*$, the simulator sets $r_i = \tilde{r}_i - f(\ell)M_{i,1}a/(\tau - \tau^*)$ so that the residual $b(\tau - \tau^*)$ term absorbs the unknown contribution $M_{i,1}f(\ell)ab$ in α_ℓ. Consequently, all key components remain computable with the correct distribution. The challenge ciphertexts are generated exactly as in the single-authority proof, embedding the DBDH challenge through $s_j = c$ at the designated hybrid step. The hybrid sequence over the B challenge ciphertexts is unchanged. Each transition replaces one masking component $C_{j,0}$ with a uniform target-group element and is reducible to a single DBDH instance. In the final hybrid, the adversary's view is independent of the challenge bit. Therefore, any coalition of fewer than T authorities learns no information about the challenge batch, even when combined with adaptive access to honest key shares for non-challenge tags and batches. The threshold construction thus preserves the security and efficiency properties of the base scheme while eliminating the single point of trust.

3 Preliminaries

We briefly recall the cryptographic preliminaries that are required in our construction.

3.1 Type-3 Bilinear Map [12]

Definition 1 (Type-3 Bilinear Map). *Consider three multiplicative cyclic groups $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ having the properties: $\mathbb{G}_1 \neq \mathbb{G}_2$ with no efficiently computable isomorphism from $\mathbb{G}_1$ to $\mathbb{G}_2$ and all three groups have same order p. Here, p represents a large prime such that its length is at least 2^λ. Assume that a random element g_1 generates the first source group $\mathbb{G}_1$ and another random element g_2 generates the second source group $\mathbb{G}_2$. A function $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is called a Type-3 Bilinear Map (also designated as T3-map), if it satisfies the following three fundamental properties.*

1. ***Bilinearity:*** *For all $X \in_R \mathbb{G}_1$, $Y \in_R \mathbb{G}_2$ and $a, b \in_R \mathbb{Z}_p^*$, it holds that $e(X^a, Y^b) = e(X, Y)^{ab}$.*
2. ***Non-degenerate:*** *There exists generators $g_1 \in_R \mathbb{G}_1$ and $g_2 \in_R \mathbb{G}_2$ such that $e(g_1, g_2) \neq 1_{\mathbb{G}_T}$. That means, the element $e(g_1, g_2)$ is a generator of the target group $\mathbb{G}_T$.*
3. ***Computability:*** *The pairing $e(g_1, g_2)$ can be computed efficiently.*

The tuple $\mathbb{G} = \big(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e\big)$ is called a prime order Type-3 bilinear map (or T3-map) system.

3.2 Type-3 Bilinear Hard Problem

Our proposed protocol's security is derived depending on the Decisional Bilinear Diffie-Hellman Type-3 (DBDH) hard problem [13], which can be described as follows.

- **Input:** The input instance $\big\langle Z = \big(\mathbb{G} = \big(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e\big),\ g_1,\ g_2,\ [a]_1, [a]_2,\ [b]_1, [b]_2,\ [c]_1\big),\ U\big\rangle$. Here, a, b and c are randomly chosen from $\mathbb{Z}_p^*$ and U is either $[abc]_T$ or a random element $X \in_R \mathbb{G}_T$.
- **Output:** The following two outputs

$$\begin{cases} 0 & \text{if } U = [abc]_T \\ 1 & \text{otherwise} \end{cases}$$

Definition 2 (DBDH Assumption). *The DBDH assumption holds if the advantage $\mathsf{Adv}_{\mathcal{A}}^{DBDH}(1^\lambda)$, which is given below, of any PPT adversary $\mathcal{A}$ in solving the above mentioned hard problem is at most a negligible quantity.*

$$\mathsf{Adv}_{\mathcal{A}}^{DBDH}(1^\lambda) = \Big| Pr\Big[\mathcal{A}(Z, U = [abc]_T) = 1\Big] - Pr\Big[\mathcal{A}(Z, U = X) = 1\Big] \Big|$$

3.3 Access Structures and Linear Secret-Sharing Schemes

The definition of access structures and linear secret-sharing schemes (LSSS) are stated as follows [4, 23].

Definition 3 (Access Structures [23]). *Let $\mathcal{U} = \{A_1, A_2, \ldots, A_n\}$ be the set of all attributes that a system can accommodate. An access structure on $\mathcal{U}$ is a collection $\mathbb{L}$ of non-empty subsets of attributes, i.e., $\mathbb{L} \subseteq 2^{\{A_1, A_2, \ldots, A_n\}} \setminus \{\emptyset\}$. The sets belonging to $\mathbb{L}$ are designated as the authorized sets, whereas the sets not belonging to $\mathbb{L}$ are designated as the unauthorized sets. Furthermore, a collection $\mathbb{A} \subseteq 2^{\{A_1, A_2, \ldots, A_n\}} \setminus \{\emptyset\}$ is called monotone access structure if $\forall\ B,\ C \in \mathbb{A}$ the following condition holds.*

$$\text{if } B \in \mathbb{A} \text{ and } B \subseteq C, \text{ then } C \in \mathbb{A}$$

Definition 4 (Linear Secret-Sharing Schemes (LSSS) [4, 23]). *Let $\mathcal{U} = \{A_1, A_2, \ldots, A_n\}$ be the attribute universe and p ($\geq 2^\eta$) be a large prime. A secret-sharing scheme, denoted by SSS, with domain of secrets $\mathbb{Z}_p$ and realizing access structures on $\mathcal{U}$ is said to be linear over $\mathbb{Z}_p$ if it satisfied the following two conditions.*

1. *Let s be a randomly chosen secret from $\mathbb{Z}_p$. For each attribute $A_i \in \mathcal{U}$, the shares corresponding to s form a vector over $\mathbb{Z}_p$.*
2. *For each access structure $\mathbb{A}$ on the attribute universe $\mathcal{U}$, there exists a label matrix $M(M, \rho)$ that satisfies the following properties: Here, $M \in \mathbb{Z}_p^{d \times n}$ is the share-generating matrix and $\rho : [d] \to \mathcal{U}$ represents the labeling function which labels the rows of matrix M with attributes from $\mathcal{U}$. We first choose random exponents $r_2, r_3, \ldots, r_n$ from $\mathbb{Z}_p$ to construct a column vector $\boldsymbol{v} = (s, r_2, \ldots, r_n)^T$. To generate the shares, we compute the vector $M\boldsymbol{v} \in \mathbb{Z}_p^{d \times 1}$ of d shares corresponding to the secret $s \in \mathbb{Z}_p$. For each $j \in [d]$, the share $(M\boldsymbol{v})_j$ corresponds to the attribute $\rho(j) \in \mathcal{U}$. Here, the label matrix $M(M, \rho)$ is denoted as the policy associated with the access structure $\mathbb{A}$.*

Following the work of Beimel [4], we emphasize that each LSSS should satisfy two requirements: (a) Reconstruction: each authorized set can reconstruct the secret and (b) Privacy: any unauthorized set cannot reveal any partial information about the secret.

Remark 1. Let $\mathbb{L}$ denote an authorized set for the access structure $\mathbb{A}$ encoded by the policy (M, ρ). Then, assume that $\mathcal{I}$ be the set of rows whose labels are in $\mathbb{L}$, *i.e.,* $\mathcal{I} = \{i | i \in [d] \wedge \rho(i) \in \mathbb{L}\}$. The reconstruction requirement asserts that the vector $\mathbf{1} = (1, 0, \ldots, 0)$ is in the span of rows of M indexed by $\mathcal{I}$. This means there exist constants $\{\nu_i\}_{i \in \mathcal{I}}$ in $\mathbb{Z}_p$ such that for any valid shares $\{\lambda_i = (M\boldsymbol{v})_i\}_{i \in \mathcal{I}}$ of a secret s according to SSS, it is true that: $\sum_{i \in \mathcal{I}} \nu_i \lambda_i = s$. Additionally, it has been proved in [4,23] that the constants $\{\nu_i\}_{i \in \mathcal{I}}$ can be found in time polynomial in the size of the share-generating matrix M.
On the other hand, for unauthorized set of attributes $\mathbb{L}'$ no such constants $\{\nu_i\}_{i \in \mathcal{I}}$ exist. Moreover, in this case it is also true that if $\mathcal{I}' = \{i | i \in [d] \wedge$

$\rho(i) \in \mathbb{L}^{'}\}$, there exist a vector $\boldsymbol{\nu} \in \mathbb{Z}_p^{1\times n}$, such that its first component ν_1 is any non-zero element in $\mathbb{Z}_p$ and $\langle \boldsymbol{M}_i, \boldsymbol{\nu}\rangle = 0$ for all $i \in \mathcal{I}^{'}$, where $\boldsymbol{M}_i = (M_{i,1}, M_{i,2}, \ldots, M_{i,n})$ represents the i-th row of M.

3.4 Shamir's (t, n)-Threshold Secret Sharing [25]

The system involves a dealer $\mathcal{D}$ and n participants, which are $\{\mathcal{P}_1, \mathcal{P}_2, \ldots, \mathcal{P}_n\}$. The protocol consists of two algorithms - (Share, Reconstruct), which are detailed below.

- Share. Suppose $\mathcal{D}$ wants to share a secret a_0 in such a way that to retrieve a_0 at least t authenticated participants out of the total number of n participants need to collude. It first chooses a large prime number d in between n and $2n$. It then randomly selects $(a_0, a_1, \ldots, a_{t-1}) \in \mathbb{Z}_d^t$, where a_0 represents the secret. It then sets a $(t-1)$ degree polynomial $f(x) = a_0 + a_1 x + \ldots + a_{t-1}x^{t-1}$. It randomly picks n non-zero distinct elements $x_1, x_2, \ldots, x_n$ from $\mathbb{Z}_d$ and makes them publicly available. It finally computes $\{f(x_i) : i = 1, 2 \ldots, n\}$ for sending it to the participants $\{\mathcal{P}_i : i = 1, 2 \ldots, n\}$ through a secure classical communication channel between them.

- Reconstruct. Suppose any t participants, out of n participants, want to retrieve the secret a_0. Without loss of generality, we assume that the set $\mathcal{S}$=$\{\mathcal{P}_i : i \in \mathcal{I}_\mathcal{S}\}$ of t participants, where $\mathcal{I}_\mathcal{S}$ represents the index set for $\mathcal{S}$ with $|\mathcal{I}_\mathcal{S}| = t$, collude together to reconstruct a_0. They execute the following.

$$a_0 = \sum_{i=1}^{t} f(x_i) \prod_{\substack{j\neq i \\ 1\leq j\leq t}} \frac{x_j}{x_j - x_i} \pmod d$$

4 Batched Attribute-based Encryption

In this section, we present the formal system model for (key-policy) Batched Attribute-Based Encryption (B-ABE). We first define the syntax of the scheme, followed by its correctness and succinctness requirements, and its formal security notion. We then show the construction of B-ABE protocol and its security analysis through hybrid arguments.

Syntax. A (key-policy) batched attribute-based encryption scheme $\prod_{\mathsf{B\text{-}ABE}}$ is a tuple of efficient algorithms $\prod_{\mathsf{B\text{-}ABE}}$ = (Setup, BatchKeyGen, Enc, Digest, BatchDec) with the following syntax:

Setup$(1^\lambda, 1^N) \to$ (mpk, msk): On input the security parameter $\lambda \in \mathbb{N}$ and an upper bound N on the attribute universe, the setup algorithm outputs a master public key mpk and a master secret key msk. We assume that the public parameters implicitly define the message space $\mathcal{M}$, the attribute universe $[N]$, and the tag space Γ associated with the scheme.

$\mathsf{Enc}(\mathsf{mpk}, x, \mathsf{tg}, m) \to \mathsf{ct}$: On input the master public key mpk, an attribute set $x \subseteq [N]$, a batch label $\mathsf{tg} \in \Gamma$, and a message $m \in \mathbb{G}_T$, the encryption algorithm outputs a ciphertext ct.

$\mathsf{Digest}(\mathcal{X}, \mathsf{tg}, \mathsf{mpk}) \to d \in \mathbb{Z}_p$: On input a collection of attribute sets $\mathcal{X} = \{x_1, \ldots, x_B\}$, a batch label $\mathsf{tg} \in \Gamma$, and the master public key mpk, the digest algorithm outputs a digest d. This algorithm is deterministic.

$\mathsf{BatchKeyGen}(\mathsf{msk}, (M, \rho), \mathsf{tg}, d) \to \mathsf{sk}$: On input the master secret key msk, an access structure (M, ρ), a batch label $\mathsf{tg} \in \Gamma$, and a digest d, the key-generation algorithm outputs a secret batch key sk associated with (M, ρ), tg, and d.

$\mathsf{BatchDec}(\mathsf{mpk}, \mathsf{sk}, \mathsf{ct}_1, \ldots, \mathsf{ct}_B) \to \{m_1, \ldots, m_B\}$ or $\perp$: On input the master public key mpk, a secret batch key sk, and ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_B$, the decryption algorithm outputs the corresponding messages $\{m_1, \ldots, m_B\}$ if decryption succeeds, and otherwise outputs the special symbol $\perp$. This algorithm is deterministic.

We require $\prod_{\mathsf{B\text{-}ABE}}$ satisfying the following properties:

- **Correctness:** For all $\lambda, N, B \in \mathbb{N}$, all $(\mathsf{mpk}, \mathsf{msk})$ in support of $\mathsf{Setup}(1^\lambda, 1^N)$, all access structures (M, ρ), all attribute sets $x_1, \ldots, x_B \subseteq [N]$ authorized for (M, ρ), all tags $\mathsf{tg} \in \Gamma$, and all messages $m_1, \ldots, m_B \in \mathbb{G}_T$, if we define $\mathcal{X} = \{x_1, \ldots, x_B\}$, then we have

$$\Pr\left[\begin{array}{r|l} \mathsf{BatchDec}(\mathsf{mpk}, \mathsf{sk}, \mathsf{ct}_1, \ldots, \mathsf{ct}_B) & crs \leftarrow \mathsf{Setup}(1^\lambda), \\ = \{m_1, \ldots, m_B\} & (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(crs), \\ & \sigma \leftarrow \mathsf{Sign}(m, \mathsf{sk}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

- **Succinctness:** The size of the batch secret key $|\mathsf{sk}|$ depends only on the size of the access structure (M, ρ), the length of the tag $|\mathsf{tg}|$, and the digest length $|d|$, and is independent of the batch size B.
- **Adaptive Security:** For a security parameter λ, an attribute universe size N, a batch size bound B, a bit $\beta \in \{0, 1\}$, and an adversary $\mathcal{A}$, the security game between the challenger and $\mathcal{A}$ proceeds as follows:
  - **Setup.** The challenger runs $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^N)$ and sends $(1^\lambda, 1^N, 1^B, \mathsf{mpk})$ to $\mathcal{A}$.
  - **Phase 1 (pre-challenge key queries).** $\mathcal{A}$ adaptively issues key queries of the form $((M, \rho), \mathsf{tg}, \mathcal{X})$ where (M, ρ) is an LSSS policy, $\mathsf{tg} \in \{0, 1\}^*$, and $\mathcal{X} = \{x_1, \ldots, x_{B'}\}$ is a collection of at most B attribute sets. The challenger responds with $\mathsf{sk} \leftarrow \mathsf{BatchKeyGen}(\mathsf{msk}, (M, \rho), \mathsf{tg}, \mathsf{Digest}(\mathcal{X}, \mathsf{tg}, \mathsf{mpk}))$, and returns sk to $\mathcal{A}$.
  - **Challenge.** $\mathcal{A}$ outputs a challenge tag $\mathsf{tg}^* \in \{0, 1\}^*$, a challenge collection of attribute sets $\mathcal{X}^* = \{x_1^*, \ldots, x_B^*\}$ with $x_b^* \subseteq [N]$, and a pair of equal-length message vectors $\{(m_0^{(b)}, m_1^{(b)})\}_{b \in [B]}$ with $m_0^{(b)}, m_1^{(b)} \in \mathbb{G}_T$. The challenger computes $\mathsf{ct}_b^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, x_b^*, \mathsf{tg}^*, m_\beta^{(b)})$ for each $b \in [B]$ and returns $(\mathsf{ct}_1^*, \ldots, \mathsf{ct}_B^*)$.

– **Phase 2 (post-challenge key queries).** $\mathcal{A}$ continues to issue key queries as in Phase 1.

– **Guess.** $\mathcal{A}$ outputs a bit $\beta' \in \{0,1\}$, which is the output of the experiment.

An adversary $\mathcal{A}$ is *admissible* if every key query $((M,\rho),\mathsf{tg},\mathcal{X})$ (in both Phase 1 and Phase 2) satisfies the following condition: if $\mathsf{tg} = \mathsf{tg}^*$, then every challenge attribute set $x_b^* \in \mathcal{X}^*$ is unauthorized for (M,ρ).

We say that B-ABE scheme is *adaptively secure* if for all polynomials $B = B(\lambda)$ and all PPT admissible adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{adp}}(\lambda) := \left|\Pr[\beta' = \beta] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

Selective Security: The selective security model is identical to the adaptive security game, except that the adversary must declare the challenge attribute sets and the challenge batch label tg^* at the beginning of the security game, before seeing the public parameters generated by Setup. Observe that selective security can be amplified to adaptive security using standard complexity-leveraging techniques [7].

4.1 Construction of Batched Attribute-Based Encryption

Let BG be a Type-3 bilinear group generator and $H : \{0,1\}^* \to \mathbb{Z}_p$ a random oracle. We construct B-ABE protocol with the following algorithms.

– $\mathsf{Setup}(1^\lambda, 1^N)$**:** On input the security parameter λ and the size N of the attribute universe, the setup algorithm first generates bilinear-group parameters $(\mathbb{G}_1,\mathbb{G}_2,\mathbb{G}_T,p,g_1,g_2,e) \leftarrow \mathsf{BG}(1^\lambda)$. It then samples random exponents $\alpha, v, h \xleftarrow{\$} \mathbb{Z}_p$ and $u_1,\ldots,u_N \xleftarrow{\$} \mathbb{Z}_p$. The algorithm outputs the master public key mpk and master secret key msk defined as

$$\mathsf{mpk} = \left(g_1,\, g_2,\, [v]_1,\, [h]_1,\, \{[u_i]_1\}_{i\in[N]},\, [\alpha]_T\right),\ \mathsf{msk} = \left(\alpha,\, v,\, h,\, \{u_i\}_{i\in[N]}\right).$$

– $\mathsf{Enc}(\mathsf{mpk}, x, \mathsf{tg}, m)$**:** To encrypt a message m under an attribute set $x \subseteq [N]$ and a batch label tg, the encryption algorithm first computes $\tau \leftarrow H(\mathsf{tg})$, and samples a random exponent $s \xleftarrow{\$} \mathbb{Z}_p$. It then outputs a ciphertext $\mathsf{ct} = (C_0, C_1, C_2, \{C_j\}_{j\in x}, \mathsf{tg}, x)$, where

$$C_0 = m \cdot [\alpha s]_T,\ C_2 = [(v + h\tau)s]_1,\ C_1 = [s]_1,\ C_j = [u_j s]_1\ (j \in x).$$

– $\mathsf{Digest}(\mathcal{X}, \mathsf{tg}, \mathsf{mpk})$**:** On input a collection of attribute sets $\mathcal{X} = \{x_1,\ldots,x_B\}$, tg, and mpk, the digest algorithm constructs the batch descriptor $\mathsf{desc} = \left(\mathsf{tg},\, B,\, x_1,\ldots,x_B,\, [\alpha]_T\right)$ and outputs the digest value $d = H(\mathsf{desc})$.

– $\mathsf{BatchKeyGen}(\mathsf{msk}, (M,\rho), \mathsf{tg}, d)$**:** Let $M \in \mathbb{Z}_p^{\ell_0 \times k}$. Given msk, an access structure (M,ρ), tg, and a digest value d, the batch key-generation algorithm first

computes $\tau \leftarrow H(\mathsf{tg})$. It then samples a random vector $\boldsymbol{y} = (\alpha, y_2, \ldots, y_k)^\top \xleftarrow{\$} \mathbb{Z}_p^k$, and computes $\lambda_i = M_i \boldsymbol{y}$ for every $i \in [\ell_0]$. For each row index $i \in [\ell_0]$, the algorithm samples $r_i, t_i \xleftarrow{\$} \mathbb{Z}_p$, and defines

$$K_{i,0} = [\lambda_i + (v + h\tau) r_i - u_{\rho(i)} t_i]_2,\ K_{i,1} = [r_i]_2,\ K_{i,2} = [t_i]_2.$$

Finally, it outputs the batch secret key $\mathsf{sk} = \left(\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i \in [\ell_0]}, \mathsf{tg}, d\right)$.

– $\mathsf{BatchDec}(\mathsf{mpk}, \mathsf{sk}, \mathsf{ct}_1, \ldots, \mathsf{ct}_B)$**.** Given the master public key mpk, the batch secret key $\mathsf{sk} = \left(\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i \in [\ell_0]}, \mathsf{tg}_{\mathsf{sk}}, d_{\mathsf{sk}}\right)$, and a collection of ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_B$, the batch decryption algorithm first checks that all ciphertexts are associated with the batch label $\mathsf{tg}_{\mathsf{sk}}$. It then extracts the corresponding attribute sets $\mathcal{X} = \{x_1, \ldots, x_B\}$, and recomputes the digest value

$$d' \leftarrow \mathsf{Digest}(\mathcal{X}, \mathsf{tg}_{\mathsf{sk}}, \mathsf{mpk}).$$

If the equality $d' = d_{\mathsf{sk}}$ does not hold, the algorithm aborts and outputs $\perp$. Next, for every $b \in [B]$, the algorithm parses the ciphertext

$$\mathsf{ct}_b = (C_{b,0}, C_{b,1}, C_{b,2}, \{C_{b,j}\}_{j \in x_b}, \mathsf{tg}, x_b).$$

It verifies that the attribute set x_b satisfies the access policy (M, ρ) associated with the secret key; otherwise, it outputs $\perp$.
Define $R_b = \{i \in [\ell_0] : \rho(i) \in x_b\}$. Using the authorized set x_b together with the LSSS matrix (M, ρ), the algorithm derives reconstruction coefficients $\{\omega_i\}_{i \in R_b}$ satisfying

$$\sum_{i \in R_b} \omega_i M_i = \boldsymbol{e}_1^\top,$$

which ensures that

$$\sum_{i \in R_b} \omega_i \lambda_i = \alpha.$$

For each index $i \in R_b$, the algorithm evaluates

$$D_{b,i} = e(C_{b,1}, K_{i,0}) \cdot e(C_{b,2}, K_{i,1})^{-1} \cdot e(C_{b,\rho(i)}, K_{i,2}),$$

where the component $C_{b,\rho(i)}$ is well-defined since $\rho(i) \in x_b$. The algorithm then combines these terms according to

$$D_b = \prod_{i \in R_b} D_{b,i}^{\omega_i},$$

and recovers the plaintext message as $m_b = \frac{C_{b,0}}{D_b}$. After decrypting all ciphertexts, the algorithm outputs the message set $\{m_1, \ldots, m_B\}$.

Remark 2 (Digest binding). The digest d is stored in sk and verified against a freshly recomputed $d' = \mathsf{Digest}(\mathcal{X}, \mathsf{tg}, \mathsf{mpk})$ before any decryption attempt. Because d encodes $(\mathsf{tg}, B, x_1, \ldots, x_B, [\alpha]_T)$, this check ensures that the key is

only used on the exact batch for which it was issued. Two keys issued for the same tag tg but different batches carry different d values and hence cannot be interchanged without detection. This is the ABE analogue of the polynomial digest in batched IBE [16], where the algebraic structure of $[F_S(\tau)]_2$ achieves the same binding.

Theorem 1 (Correctness). *The Construction 4.1 for B-ABE satisfies correctness and succinctness.*

Proof. The digest check passes by construction. For each $b \in [B]$ and row $i \in R_b$ (where $\rho(i) \in x_b$), write $A = u_{\rho(i)}$ and $\Delta = v + h\tau$. We compute the following.

$$\begin{aligned}
&e(C_{b,1}, K_{i,0}) = e([s_b]_1, [\lambda_i + \Delta r_i - At_i]_2) \ = [s_b\lambda_i + s_b\Delta r_i - s_bAt_i]_T,\\
&e(C_{b,2}, K_{i,1})^{-1} = e([\Delta s_b]_1, [r_i]_2)^{-1} \ = [-s_b\Delta r_i]_T,\\
&e(C_{b,\rho(i)}, K_{i,2}) = e([As_b]_1, [t_i]_2) = [s_bAt_i]_T.
\end{aligned}$$

From the above equation,

$$\begin{aligned}
D_{b,i} &= e(C_{b,1}, K_{i,0}) \cdot e(C_{b,2}, K_{i,1})^{-1} \cdot e(C_{b,\rho(i)}, K_{i,2})\\
&= [s_b\lambda_i + s_b\Delta r_i - s_bAt_i - s_b\Delta r_i + s_bAt_i]_T\\
&= [s_b\lambda_i]_T.
\end{aligned}$$

$$\text{By } \mathsf{LSSS} \text{ reconstruction: } D_b = \prod_{i \in R_b} D_{b,i}^{\omega_i} \ = \Big[s_b \sum_{i \in R_b} \omega_i\lambda_i\Big]_T \ = [s_b\alpha]_T.$$

Therefore, $\frac{C_{b,0}}{D_b} = m_b \cdot \frac{[\alpha s_b]_T}{[\alpha s_b]_T} = m_b$.
We now argue succinctness. The batch secret key consists of the tuple $\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i\in[\ell]}$, together with the batch label tg and the digest value d. Thus, the size of the secret key grows linearly only with the number of rows ℓ in the access matrix M, and is completely independent of the batch size B. Consequently, Construction 4.1 satisfies succinctness.

Theorem 2 (Selective Security). *Let $B = B(\lambda)$ be any polynomial. Assuming DBDH holds, Construction 4.1 is selectively secure for batch size B.*

Proof. Let $\mathcal{A}$ be an efficient admissible adversary in the selective-security experiment for Construction 4.1 with batch size B. We define a sequence of hybrid games indexed by a challenge bit $\beta \in \{0, 1\}$.

- $\mathsf{Game}_0^{(\beta)}$: the real selective-security experiment $\mathsf{Sel}_\beta(\mathcal{A})$.
- $\mathsf{Game}_1^{(\beta)}$: identical to $\mathsf{Game}_0^{(\beta)}$, except that the component $C_{1,0}$ of the first challenge ciphertext is replaced by

  $$C_{1,0} = m_\beta^{(1)} \cdot [z_1]_T,$$

  where $z_1 \xleftarrow{\$} \mathbb{Z}_p$ is chosen uniformly at random. All remaining challenge ciphertexts are generated honestly.

– $\mathsf{Game}_{1+j}^{(\beta)}$: For $j = 1, \ldots, B-1$, $\mathsf{Game}_{1+j}^{(\beta)}$ is defined from $\mathsf{Game}_{j}^{(\beta)}$ by additionally replacing the component $C_{j+1,0}$ with

$$C_{j+1,0} = m_{\beta}^{(j+1)} \cdot [z_{j+1}]_T,$$

where $z_{j+1} \xleftarrow{\$} \mathbb{Z}_p$ is sampled independently and uniformly at random.

– $\mathsf{Game}_{B}^{(\beta)}$: all challenge components $C_{b,0}$, for $b \in [B]$, are replaced by independent uniform elements of $\mathbb{G}_T$.

Lemma 1. *For every $j \in [B]$ and every $\beta \in \{0,1\}$, there exists a probabilistic polynomial-time reduction $\mathcal{B}_{j,\beta}$ such that*

$$\left|\Pr[\mathsf{Game}_{j-1}^{(\beta)}(\mathcal{A}) = 1] - \Pr[\mathsf{Game}_{j}^{(\beta)}(\mathcal{A}) = 1]\right| \leq \mathsf{Adv}_{\mathcal{B}_{j,\beta}}^{\mathsf{DBDH}}(\lambda).$$

Proof. The reduction $\mathcal{B}_{j,\beta}$ is given a DBDH challenge instance $(\mathsf{par}, \mathcal{T})$ and must distinguish whether $\mathcal{T} = [abc]_T$ or $\mathcal{T} \xleftarrow{\$} \mathbb{G}_T$. It simulates the view of $\mathcal{A}$ as follows.

Setup. After receiving the adversary's commitment $(x_1^*, \ldots, x_B^*, tg^*)$, the reduction samples

$$\tilde{\alpha}, \tilde{v}, \tilde{h} \xleftarrow{\$} \mathbb{Z}_p, \qquad u_1, \ldots, u_N \xleftarrow{\$} \mathbb{Z}_p,$$

and programs the random oracle so that $H(tg^*) = \tau^*$ for a freshly sampled value $\tau^* \xleftarrow{\$} \mathbb{Z}_p$. The master secret key is defined implicitly by

$$\alpha = ab + \tilde{\alpha},\ v = -b\tau^* + \tilde{v},\ h = b + \tilde{h}.$$

Although the simulator does not know a or b, all public parameters remain computable:

$$[v]_1 = g_1^{\tilde{v}} \cdot [b]_1^{-\tau^*},\ [\alpha]_T = e([a]_1, [b]_2) \cdot e(g_1, g_2)^{\tilde{\alpha}},$$
$$[h]_1 = g_1^{\tilde{h}} \cdot [b]_1, \quad [u_i]_1 = g_1^{u_i},\ \ i \in [N].$$

The resulting master public key is then given to $\mathcal{A}$.

Key-generation queries. Consider a query $((M, \rho), tg, d)$ with $tg \neq tg^*$. Let $\tau = H(tg)$. Since $\tau \neq \tau^*$, defining $\Delta = \tau - \tau^*$ yields $v + h\tau = \tilde{v} + \tilde{h}\tau + b\Delta$. The simulator samples $y_2, \ldots, y_k \xleftarrow{\$} \mathbb{Z}_p$, and for every row $i \in [\ell_0]$ chooses $\tilde{r}_i, \tilde{t}_i \xleftarrow{\$} \mathbb{Z}_p$, while implicitly setting

$$r_i = \tilde{r}_i - \frac{M_{i,1}a}{\Delta},\ \ t_i = \tilde{t}_i.$$

Writing

$$\lambda_i = M_{i,1}(ab + \tilde{\alpha}) + \sum_{l \geq 2} M_{i,l} y_l,$$

the exponent of $K_{i,0}$ becomes

$$\lambda_i + (v + h\tau)r_i - u_{\rho(i)}t_i = \underbrace{M_{i,1}\tilde{\alpha} + \sum_{l\geq 2} M_{i,l}y_l + (\tilde{v} + \tilde{h}\tau)\tilde{r}_i - u_{\rho(i)}\tilde{t}_i}_{\sigma_i} + a\underbrace{\left(-\frac{(\tilde{v} + \tilde{h}\tau)M_{i,1}}{\Delta}\right)}_{\gamma_i} + b\underbrace{(\Delta\tilde{r}_i)}_{\delta_i'}.$$

Hence

$$K_{i,0} = g_2^{\sigma_i} \cdot [a]_2^{\gamma_i} \cdot [b]_2^{\delta_i'},\ K_{i,1} = [\tilde{r}_i]_2 \cdot [a]_2^{-M_{i,1}/\Delta},\quad K_{i,2} = [\tilde{t}_i]_2.$$

All components are efficiently computable from the challenge instance. Moreover, since $(\tilde{r}_i, \tilde{t}_i)$ are uniformly random, the induced distribution of (r_i, t_i) is identical to that of a genuine key.

Challenge ciphertexts. The simulator embeds the DBDH challenge into the j-th ciphertext by setting $s_j = c$. It is observed that

$$\begin{aligned} v + h\tau^* &= (-b\tau^* + \tilde{v}) + (b + \tilde{h})\tau^* \\ &= \tilde{v} + \tilde{h}\tau^*, \end{aligned}$$

so the dependence on b vanishes. For every $b < j$, the simulator samples fresh values $s_b, z_b \xleftarrow{\$} \mathbb{Z}_p$ and sets

$$\begin{aligned} &C_{b,0} = m_\beta^{(b)} \cdot [z_b]_T,\ C_{b,1} = [s_b]_1, \\ &C_{b,2} = [s_b]_1^{\tilde{v}+\tilde{h}\tau^*},\quad C_{b,k} = [s_b]_1^{u_k},\ k \in x_b^*. \end{aligned}$$

For the critical ciphertext $b = j$, the simulator defines

$$\begin{aligned} &C_{j,1} = [c]_1,\ C_{j,2} = [c]_1^{\tilde{v}+\tilde{h}\tau^*}, C_{j,k} = [c]_1^{u_k},\quad k \in x_j^*, \\ &C_{j,0} = m_\beta^{(j)} \cdot T \cdot e([c]_1, g_2)^{\tilde{\alpha}}. \end{aligned}$$

If $\mathcal{T} = [abc]_T$, then $C_{j,0} = m_\beta^{(j)} \cdot [abc]_T \cdot [c\tilde{\alpha}]_T = m_\beta^{(j)} \cdot [\alpha c]_T$, which is distributed exactly as in a real encryption.
Conversely, if $\mathcal{T}$ is uniform in $\mathbb{G}_T$, then $C_{j,0}$ is also uniformly random in $\mathbb{G}_T$, independently of the message. For every $b > j$, the simulator generates the following ciphertext components using a fresh exponent $s_b \xleftarrow{\$} \mathbb{Z}_p$:

$$\begin{aligned} &C_{b,0} = m_\beta^{(b)} \cdot [\alpha s_b]_T = m_\beta^{(b)} \cdot e([a]_1, [b]_2)^{s_b} \cdot e(g_1, g_2)^{\tilde{\alpha} s_b}, \\ &C_{b,1} = [s_b]_1,\ C_{b,2} = [s_b]_1^{\tilde{v}+\tilde{h}\tau^*},\ C_{b,k} = [s_b]_1^{u_k}. \end{aligned}$$

All terms are efficiently computable from the challenge instance and the simulator's randomness.

Analysis. If $\mathcal{T} = [abc]_T$, then the simulation is distributed exactly as game $\mathsf{Game}_{j-1}^{(\beta)}$. If instead $\mathcal{T}$ is uniformly random in $\mathbb{G}_T$, the resulting distribution coincides with game $\mathsf{Game}_{j}^{(\beta)}$. Therefore,

$$\left|\Pr[\mathsf{Game}_{j-1}^{(\beta)} = 1] - \Pr[\mathsf{Game}_{j}^{(\beta)} = 1]\right| \leq \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,\beta}}(\lambda),$$

which completes the proof.

Lemma 2. $\Pr[\mathit{Game}_B^{(0)}(\mathcal{A}) = 1] = \Pr[\mathit{Game}_B^{(1)}(\mathcal{A}) = 1]$.

Proof. In the game $\mathsf{Game}_B^{(\beta)}$, every challenge component is of the form $C_{b,0} = m_\beta^{(b)} \cdot [z_b]_T$, where $z_b \xleftarrow{\$} \mathbb{Z}_p$ is chosen uniformly at random. Consequently, each $C_{b,0}$ is itself uniformly distributed in $\mathbb{G}_T$ and independent of the challenge bit β. All remaining ciphertext components and all other values revealed during the game are generated independently of β. Therefore, the adversary's view is identical in $\mathsf{Game}_B^{(0)}$ and $\mathsf{Game}_B^{(1)}$, implying

$$\Pr[\mathsf{Game}_B^{(0)}(\mathcal{A}) = 1] = \Pr[\mathsf{Game}_B^{(1)}(\mathcal{A}) = 1].$$

By the game sequence and triangle inequality, we have the following:

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{sel}}_{\mathcal{A}} &= |\Pr[\mathsf{Game}_0^{(0)} = 1] - \Pr[\mathsf{Game}_0^{(1)} = 1]| \\
&\leq |\Pr[\mathsf{Game}_0^{(0)} = 1] - \Pr[\mathsf{Game}_B^{(0)} = 1]| \\
&+ |\Pr[\mathsf{Game}_B^{(0)} = 1] - \Pr[\mathsf{Game}_B^{(1)} = 1]| \\
&+ |\Pr[\mathsf{Game}_B^{(1)} = 1] - \Pr[\mathsf{Game}_0^{(1)} = 1]| \\
&\leq \sum_{j=1}^{B} \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,0}} + 0 + \sum_{j=1}^{B} \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,1}} \\
&\leq 2B \cdot \mathsf{Adv}^{\mathsf{DBDH}}.
\end{aligned}$$

5 Threshold Batched Attribute-based Encryption

In this section, we extend the proposed $\mathsf{B\text{-}ABE}$ scheme to the threshold setting. We begin by presenting the syntax of the resulting threshold construction, followed by its security model, concrete instantiation, and security analysis.

Syntax of Threshold Batched ABE. A Threshold Batched Attribute-Based Encryption ($\mathsf{TB\text{-}ABE}$) scheme consists of four types of entities: a trusted dealer, encryptors, decryption authorities, and decryptors. The trusted dealer executes the KeyGen algorithm to generate and distribute key-share pairs $(\mathsf{sk}_\ell, \mathsf{pk}_\ell)_{\ell \in [L]}$ along with a master public key mpk. Encryptors use mpk to encrypt messages under designated attribute sets. Each decryption authority holds a single secret-key share and independently computes a partial decryption key. A decryptor

gathers at least T valid key shares, combines them to reconstruct the decryption capability, and subsequently decrypts a batch of ciphertexts. Formally, a TB-ABE is defined by the tuple of efficient algorithms TB-ABE = (Setup, KeyGen, Enc, Digest, CompKeyShare, VerifyKeyShare, CompKeyAgg, BatchDec), which are described below.

- $\mathsf{Setup}(1^\lambda, 1^N) \to \mathsf{pp}$. Given a security parameter λ and an upper bound N on the attribute universe, this algorithm generates the public parameters pp. The output parameters implicitly specify the message space $\mathcal{M} = \mathbb{G}_T$, the attribute universe $[N]$, and the tag space $\Gamma = \{0,1\}^\lambda$.
- $\mathsf{KeyGen}(\mathsf{pp}, 1^B, 1^L, T) \to \big(\{\mathsf{pk}_\ell, \mathsf{sk}_\ell\}_{\ell\in[L]}, \mathsf{mpk}\big)$. The trusted dealer takes as input the public parameters pp, a maximum batch size B, the number of decryption authorities L, and a threshold value $T \le L$. It outputs a master public key mpk and, for each authority $\ell \in [L]$, a public/secret key-share pair $(\mathsf{pk}_\ell, \mathsf{sk}_\ell)$.
- $\mathsf{Enc}(\mathsf{mpk}, m, x, \mathsf{tg}) \to \mathsf{ct}$. Given the master public key mpk, a message $m \in \mathbb{G}_T$, an attribute set $x \subseteq [N]$, and a batch identifier $\mathsf{tg} \in \Gamma$, the encryption algorithm produces a ciphertext ct.
- $\mathsf{Digest}(\mathsf{mpk}, \mathcal{X}, \mathsf{tg}) \to d$. On input the master public key mpk, a batch of attribute sets $\mathcal{X} = \{x_1, \ldots, x_B\}$, and a tag tg, this deterministic procedure computes a digest value $d \in \mathbb{Z}_p$ that uniquely and cryptographically binds the entire batch.
- $\mathsf{CompKeyShare}(\mathsf{sk}_\ell, (M,\rho), d, \mathsf{tg}) \to \sigma_\ell$. Given authority ℓ's secret key share sk_ℓ, an LSSS access structure (M,ρ), the batch digest d, and the associated tag tg, this algorithm generates a partial decryption key share σ_ℓ.
- $\mathsf{VerifyKeyShare}(\mathsf{pk}_\ell, (M,\rho), d, \mathsf{tg}, \sigma_\ell) \to \{0,1\}$. This deterministic verification algorithm takes as input the public key of authority ℓ, the access structure (M,ρ), the digest d, the tag tg, and a purported key share σ_ℓ. It outputs 1 if the share is valid and 0 otherwise.
- $\mathsf{CompKeyAgg}(\mathsf{mpk}, \{\sigma_\ell\}_{\ell\in U}, (M,\rho), d, \mathsf{tg}) \to \bar{\sigma}$. Given the master public key mpk, a collection of T valid key shares corresponding to a subset of authorities $U \subseteq [L]$ with $|U| = T$, together with the access structure, digest, and tag, this deterministic algorithm computes an aggregated batch decryption key $\bar{\sigma}$.
- $\mathsf{BatchDec}(\mathsf{mpk}, \bar{\sigma}, (\mathcal{X}, \mathsf{tg}), \mathsf{ct}_1, \ldots, \mathsf{ct}_B) \to \{m_1, \ldots, m_B\}$ or $\perp$. Given the master public key mpk, an aggregated decryption key $\bar{\sigma}$, the batch description $(\mathcal{X}, \mathsf{tg})$, and a collection of ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_B$, this algorithm outputs the corresponding plaintext messages if decryption succeeds; otherwise, it returns $\perp$.

A TB-ABE protocol should satisfy the following properties:

- **Completeness:** A TB-ABE scheme is said to be complete if every honestly generated key share is accepted by the verification algorithm. Formally, for all

$\lambda, N, B, L, T \in \mathbb{N}$ with $T \leq L$, all $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^N)$, all $(\{\mathsf{pk}_\ell, \mathsf{sk}_\ell\}, \mathsf{mpk}) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^B, 1^L, T)$, all (M, ρ), all tg, and all $\mathcal{X}$, it holds that

$$\Pr\left[\begin{array}{l|l} \mathsf{VerifyKeyShare}(\mathsf{pk}_\ell, & d = \mathsf{Digest}(\mathsf{mpk}, \mathcal{X}, \mathsf{tg}), \\ (M, \rho), d, \mathsf{tg}, \sigma_\ell) = 1 & \sigma_\ell \leftarrow \mathsf{CompKeyShare}(\mathsf{sk}_\ell, (M, \rho), d, \mathsf{tg}) \end{array}\right] = 1.$$

- **Correctness:** A TB-ABE scheme is correct if any collection of at least T valid key shares enables successful batch decryption of all ciphertexts whose attributes satisfy the target access policy. More precisely, for all admissible parameters, messages $m_1, \ldots, m_B \in \mathbb{G}_T$, authorized attribute sets $x_1, \ldots, x_B$, and every subset $U \subseteq [L]$ satisfying $|U| = T$, if ciphertexts are generated as $\mathsf{ct}_b \leftarrow \mathsf{Enc}(\mathsf{mpk}, m_b, x_b, \mathsf{tg})$ and valid key shares $\{\sigma_\ell\}_{\ell \in U}$ are computed and aggregated into $\bar{\sigma} = \mathsf{CompKeyAgg}(\mathsf{mpk}, \sigma_{\ell \ell \in U}, (M, \rho), d, \mathsf{tg})$, then

$$\mathsf{BatchDec}(\mathsf{mpk}, \bar{\sigma}, \mathcal{X}, \mathsf{tg}, \mathsf{ct}_1, \ldots, \mathsf{ct}_B) = \{m_1, \ldots, m_B\}.$$

- **Succinctness:** The scheme is succinct if the sizes of both individual key shares and the aggregated key share remain independent of the batch size. Specifically, there exists a polynomial $\mathrm{poly}(\cdot)$ such that:
  - each key share σ_ℓ has size $\mathrm{poly}(\lambda, \ell_0, k)$ and is independent of B;
  - the aggregated key share $\bar{\sigma}$ has size $\mathrm{poly}(\lambda, \ell_0)$ and is independent of both B and the number of participating authorities $|U|$.

  Here, ℓ_0 and k denote the number of rows and columns of the underlying LSSS matrix M, respectively.

- **Static Security Game**
  Security is defined through an indistinguishability game between a challenger and a PPT adversary $\mathcal{A}$. The adversary is allowed to statically corrupt a subset of authorities before the game begins, while retaining the ability to adaptively request key shares throughout the execution. For a security parameter λ, parameters N, B, L, T, corruption set $C \subset [L]$ with $|C| < T$, challenge bit $\beta \in \{0, 1\}$, and adversary $\mathcal{A}$, define the experiment $\mathsf{Expt}_\beta^{\mathsf{TB\text{-}ABE}}(\mathcal{A})$:
  - **Commit.** $\mathcal{A}$ declares the challenge tag tg^*, challenge attribute collection $\mathcal{X}^* = \{x_1^*, \ldots, x_B^*\}$, committee size L, threshold T, and corruption set $C \subset [L]$ with $|C| < T$.
  - **Setup.** The challenger runs $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^N)$ and $(\{\mathsf{pk}_\ell, \mathsf{sk}_\ell\}_{\ell \in [L]}, \mathsf{mpk}) \leftarrow \mathsf{KeyGen}(\mathsf{pp}, 1^B, 1^L, T)$ honestly. It gives $\big(\mathsf{pp}, \mathsf{mpk}, \{\mathsf{pk}_\ell\}_{\ell \in [L]}, \{\mathsf{sk}_\ell\}_{\ell \in C}\big)$ to $\mathcal{A}$.
  - **Phase 1 (adaptive key-share queries).** $\mathcal{A}$ adaptively submits queries $(\ell, (\mathbf{M}, \rho), \mathcal{X}, \mathsf{tg})$ with $\ell \in [L] \setminus C$, $\mathcal{X} = \{x_1, \ldots, x_{B'}\}$ for some $B' \leq B$, and $\mathsf{tg} \in \mathit{\Gamma}$. The challenger responds with $\sigma_\ell \leftarrow \mathsf{CompKeyShare}(\mathsf{sk}_\ell, (\mathbf{M}, \rho), \mathsf{Digest}(\mathsf{mpk}, \mathcal{X}, \mathsf{tg}), \mathsf{tg})$.
  - **Challenge.** $\mathcal{A}$ outputs two message vectors $\mathbf{m}_0 = (m_1^{(0)}, \ldots, m_B^{(0)})$ and $\mathbf{m}_1 = (m_1^{(1)}, \ldots, m_B^{(1)})$. The challenger returns $ct_b^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, m_b^{(\beta)}, x_b^*, \mathsf{tg}^*)$, $b \in [B]$, where the challenge attribute sets x_b^* were fixed at the commit step.

– **Phase 2.** $\mathcal{A}$ continues to submit key-share queries as in Phase 1.

– **Output.** $\mathcal{A}$ outputs a guess $\beta' \in \{0,1\}$.

Admissibility. An adversary $\mathcal{A}$ is admissible if every key-share query $(\ell, (\mathbf{M},\rho), \mathcal{X}, \mathsf{tg})$ satisfies:

– each authority $\ell \in [L] \setminus C$ is queried on the challenge tag tg^* at most once.

– if $\mathsf{tg} = \mathsf{tg}^*$, then every challenge attribute set $x_b^* \in \mathcal{X}^*$ is unauthorised for the queried policy $(\mathbf{M},\rho)$.

We say TB-ABE is static secure if for all polynomials N, B, L and all PPT admissible adversaries $\mathcal{A}$:

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{TB\text{-}ABE}}(\lambda) := \left|\Pr[\mathsf{Expt}_0^{\mathsf{TB\text{-}ABE}}(\mathcal{A}) = 1] - \Pr[\mathsf{Expt}_1^{\mathsf{TB\text{-}ABE}}(\mathcal{A}) = 1]\right| \leq \mathsf{negl}(\lambda).$$

5.1 Construction of Threshold Batched TB-ABE

Let BG be a Type-3 bilinear group generator and let $H : \{0,1\}^* \to \mathbb{Z}_p$ be a cryptographic hash function modelled as a random oracle. Our threshold batched attribute-based encryption (TB-ABE) construction extends the basic batched ABE framework by distributing the master secret among multiple authorities using Shamir secret sharing and enabling threshold-based generation of decryption capabilities. We construct TB-ABE protocol with the following algorithm.

– $\mathsf{Setup}(1^\lambda, 1^N)$**:** On input the security parameter λ and the attribute universe size N, the setup algorithm invokes the bilinear-group generator BG to obtain the public parameters

$$(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e) \leftarrow \mathsf{BG}(1^\lambda),$$

where $\mathbb{G}_1$ and $\mathbb{G}_2$ are cyclic groups of prime order p, $\mathbb{G}_T$ is the target group, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerate bilinear pairing. The algorithm then publishes the system parameters

$$\mathsf{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e, H, N),$$

where $H : \{0,1\}^* \to \mathbb{Z}_p$ denotes a cryptographic hash function modelled as a random oracle.

– $\mathsf{KeyGen}(\mathsf{pp}, 1^B, 1^L, T)$: Given the public parameters pp, the maximum batch size B, the number of authorities L, and the threshold value T, the trusted dealer first samples the master secret exponents

$$\alpha, v, h \leftarrow \mathbb{Z}_p, \qquad u_1, \ldots, u_N \leftarrow \mathbb{Z}_p.$$

To distribute the master secret among the authorities, the dealer constructs a (T, L)-Shamir secret-sharing of α. Specifically, it selects a random polynomial

$$f_\alpha(x) = \sum_{j=0}^{T-1} a_j x^j$$

of degree at most $(T-1)$ such that $f_\alpha(0) = \alpha$ (equivalently, $a_0 = \alpha$). For each authority $\ell \in [L]$, the corresponding secret share is computed as $\alpha_\ell = f_\alpha(\ell)$. The dealer then assigns to authority ℓ's the secret key $\mathsf{sk}_\ell = \big(\alpha_\ell, v, h, u_j j \in [N]\big)$, and the associated public verification key $\mathsf{pk}\ell = [\alpha_\ell]_T$. Finally, the dealer publishes the master public key

$$\mathsf{mpk} = \Big(\mathsf{pp}, [v]_1, [h]_1, \{[u_j]_1\}_{j\in[N]}, [\alpha]_T, \{\mathsf{pk}_\ell\}_{\ell\in[L]}, \{[a_j]_1\}_{j=0}^{T-1}\Big).$$

The commitments $\{[a_j]_1\}_{j=0}^{T-1}$ to the coefficients of the Shamir polynomial are included to enable public verification of the distributed secret shares, while the values $\{\mathsf{pk}_\ell\}_{\ell\in[L]}$ serve as authority-specific public verification keys during the key-share verification procedure.

Upon receiving its secret share $\mathsf{sk}_\ell = (\alpha_\ell, v, h, u_j)$, authority ℓ can independently verify the correctness of the share using the polynomial commitments contained in the master public key mpk. Specifically, it retrieves the commitment coefficients $\{[a_j]_1\}_{j=0}^{T-1}$ and computes the commitment corresponding to the polynomial evaluation at point ℓ as

$$[\alpha_\ell]_1^* = \prod_{j=0}^{T-1} [a_j]_1^{\ell^j}.$$

The authority then checks whether $g_1^{\alpha\ell} = [\alpha_\ell]_1^*$. If the equality holds, the verification succeeds; otherwise, the share is rejected. This verification procedure requires $O(T)$ exponentiations in $\mathbb{G}_1$ and is performed entirely locally, without any interaction with other authorities. A successful verification confirms that α_ℓ is a valid evaluation of the dealer-committed polynomial of degree at most $(T-1)$.

– $\mathsf{Enc}(\mathsf{mpk}, m, x, \mathsf{tg})$**:** Let mpk be the master public key, m the message to be encrypted, x the attribute set associated with the ciphertext, and tg the tag. The sender first computes $\tau = H(\mathsf{tg}) \in \mathbb{Z}_p$ and then samples a random exponent $s \xleftarrow{\$} \mathbb{Z}_p$. Using these values, the ciphertext is generated as $\mathsf{ct} = \Big(C_0, C_1, C_2, \{C_j = [u_j s]_1\}_{j\in x}, \mathsf{tg}, x\Big)$, where

$$\begin{aligned} C_0 &= m \cdot [\alpha s]_T, \quad & C_2 &= [(v + h\tau)s]_1, \\ C_1 &= [s]_1, & C_j &= [u_j s]_1 \end{aligned}$$

– $\mathsf{Digest}(\mathsf{mpk}, \mathcal{X}, \mathsf{tg})$**:** Given a set of attributes $\mathcal{X} = \{x_1, \ldots, x_{B'}\}$, where $B' = |\mathcal{X}|$, the algorithm computes the digest value as

$$d = H\big(\mathsf{tg},\, B',\, x_1, \ldots, x_{B'},\, [\alpha]_T\big) \in \mathbb{Z}_p.$$

– $\mathsf{CompKeyShare}(\mathsf{sk}_\ell, (\mathbf{M}, \rho), d, \mathsf{tg})$**:** Let the authority secret key be parsed as $\mathsf{sk}_\ell = (\alpha_\ell, v, h, \{u_j\}_{j\in[N]})$, and let $(\mathbf{M}, \rho)$ denote the access structure, where $\mathbf{M} \in \mathbb{Z}_p^{\ell_0 \times k}$ is the LSSS matrix and $\rho : [\ell_0] \to [N]$ maps each row of $\mathbf{M}$ to an

attribute index. The algorithm first computes $\tau = H(\mathsf{tg})$. It then samples a random vector

$$\mathbf{y} = (\alpha_\ell, y_2, \ldots, y_k)^\top \in \mathbb{Z}_p^k,$$

whose first component is fixed to α_ℓ and whose remaining entries are chosen uniformly at random. For each row $i \in [\ell_0]$, an LSSS share is generated as $\lambda_i = \mathbf{M}_i\mathbf{y}$. The algorithm then samples independent random values $r_i, t_i \in \mathbb{Z}_p$ for every $i \in [\ell_0]$. Using these values together with the secret parameters v, h, and $\{u_j\}_{j\in[N]}$, the key-share components are computed as

$$K_{i,0} = [\lambda_i + (v + h\tau)r_i - u_{\rho(i)}t_i]_2, \qquad K_{i,1} = [r_i]_2, \qquad K_{i,2} = [t_i]_2.$$

The algorithm further computes the auxiliary LSSS commitments

$$[y_j]_1 = g_1^{y_j}, \qquad j = 2, \ldots, k,$$

which are used to facilitate subsequent verification and reconstruction procedures. Finally, the generated key share is output as

$$\sigma_\ell = \Big(\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i\in[\ell_0]}, \{[y_j]_1\}_{j=2}^k, d\Big).$$

– $\mathsf{VerifyKeyShare}(\mathsf{pk}_\ell, (\mathbf{M}, \rho), d, \mathsf{tg}, \sigma_\ell)$**:** Let $\sigma_\ell = \big(\{K_{i,0}, K_{i,1}, K_{i,2}\}_{i\in[\ell_0]}, \{[y_j]_1\}_{j=2}^k, d'\big)$. The verification algorithm first check whether the embedded depth value d' matches the input depth d. If $d' \neq d$, the algorithm immediately rejects and outputs 0. Otherwise, it computes $\tau \leftarrow H(\mathsf{tg})$.

  For every row $i \in [\ell_0]$, the verifier reconstructs the corresponding LSSS-share commitment using the public key share pk_ℓ together with the auxiliary commitments contained in σ_ℓ:

$$[\lambda_i^*]_T = \mathsf{pk}_\ell^{M_{i,1}} \cdot \prod_{j=2}^{k} e([y_j]_1, g_2)^{M_{i,j}}.$$

  Subsequently, for each row i, it verifies the consistency of the key-share components by checking whether

$$e(g_1, K_{i,0}) = e\big([v]_1 + \tau[h]_1, K_{i,1}\big) \cdot e\big([u_{\rho(i)}]_1, K_{i,2}\big)^{-1} \cdot [\lambda_i^*]_T,$$

  where $[v]_1$, $[h]_1$, and $[u_{\rho(i)}]_1$ are obtained from the master public key mpk. The algorithm accepts and outputs 1 if all pairing equations hold; otherwise, it outputs 0.

– $\mathsf{CompKeyAgg}(\mathsf{mpk}, \{\sigma_\ell\}_{\ell\in U}, (\mathbf{M}, \rho), d, \mathsf{tg})$**:** Consider a committee $U \subseteq [L]$ of size $|U| = T$. The committee first computes the Lagrange interpolation coefficients $\{\mu_\ell^U\}_{\ell\in U}$ associated with the underlying Shamir secret-sharing scheme and evaluated at the point 0. These coefficients satisfy,

$$\sum_{\ell\in U} \mu_\ell^U \alpha_\ell = \alpha,$$

thereby enabling reconstruction of the secret exponent from any threshold subset of valid shares. Using these coefficients, the committee aggregates the verified key-share components row-wise. Specifically, for every $i \in [\ell_0]$, it computes

$$\bar{K}_{i,0} = \prod_{\ell \in U} K_{i,0}^{\mu_\ell^{\mathcal{U}}}, \qquad \bar{K}_{i,1} = \prod_{\ell \in U} K_{i,1}^{\mu_\ell^{\mathcal{U}}}, \qquad \bar{K}_{i,2} = \prod_{\ell \in U} K_{i,2}^{\mu_\ell^{\mathcal{U}}}.$$

Finally, the algorithm outputs the aggregated decryption key

$$\bar{\sigma} = \Big(\{\bar{K}_{i,0}, \bar{K}_{i,1}, \bar{K}_{i,2}\}_{i \in [\ell_0]}, d \Big).$$

– $\mathsf{BatchDec}(\mathsf{mpk}, \bar{\sigma}, \mathcal{X}, \mathsf{tg}, ct_1, \ldots, ct_B)$**:** Given the master public key mpk, the aggregated decryption key $\bar{\sigma}$, the batch access structure $\mathcal{X}$, a tag tg, and a collection of ciphertexts $\{\mathsf{ct}_b\}_{b \in [B]}$, the decryptor first parses $\bar{\sigma} = (\{\bar{K}_{i,0}, \bar{K}_{i,1}, \bar{K}_{i,2}\}_{i \in [\ell_0]}, d)$. It then recomputes $d' = \mathsf{Digest}(\mathsf{mpk}, \mathcal{X}, \mathsf{tg})$ and verifies that $d' = d$. If the equality does not hold, the algorithm outputs $\perp$, indicating that the supplied batch access structure is inconsistent with the label embedded in $\bar{\sigma}$.

For each ciphertext ct_b, where $b \in [B]$, the decryptor parses

$$ct_b = (C_{b,0}, C_{b,1}, C_{b,2}, \{C_{b,j}\}_{j \in x_b}, \mathsf{tg}, x_b)$$

and checks that the tag contained in the ciphertext matches the supplied tag tg. If this verification fails, the algorithm outputs $\perp$. Next, it determines the set $R_b = \{i \in [\ell_0] : \rho(i) \in x_b\}$ and computes reconstruction coefficients $\{\omega_i\}_{i \in R_b}$ satisfying

$$\sum_{i \in R_b} \omega_i \mathbf{M}_{i,\cdot} = \mathbf{e}_1^\top.$$

Using these coefficients, for every $i \in R_b$ the decryptor evaluates

$$D_{b,i} = e(C_{b,1}, \bar{K}_{i,0}) \cdot e(C_{b,2}, \bar{K}_{i,1})^{-1} \cdot e(C_{b,\rho(i)}, \bar{K}_{i,2}).$$

It then aggregates these values as $D_b = \prod_{i \in R_b} D_{b,i}^{\omega_i}$ and recovers the plaintext message by computing

$$m_b = \frac{C_{b,0}}{D_b}.$$

After processing all ciphertexts in the batch, the algorithm outputs the collection of recovered messages $\{m_1, \ldots, m_B\}$.

Theorem 3 (Correctness). *Construction 5.1 satisfies the correctness property of threshold batched attribute-based encryption.*

Proof. Let λ, N, B, L, and T be valid system parameters. Consider attribute sets $x_1, \ldots, x_B$, each satisfying the access structure $(\mathbf{M}, \rho)$, messages $m_1, \ldots, m_B \in \mathbb{G}_T$, and a subset $U \subseteq [L]$ of authorities such that $|U| = T$. We define $\tau = H(\mathsf{tg})$ and $\Delta = v + h\tau$.

By the reconstruction property of Shamir secret sharing, the Lagrange coefficients $\{\mu_\ell^U\}_{\ell\in U}$ satisfy

$$\sum_{\ell\in U}\mu_\ell^{\mathcal{U}}\alpha_\ell=\alpha$$

Using these coefficients, define the aggregated components

$$\bar{\lambda}_i=\sum_\ell \mu_\ell^{\mathcal{U}}\lambda_i^\ell,\qquad \bar{r}_i=\sum_\ell \mu_\ell^{\mathcal{U}} r_i^\ell,\qquad \bar{t}_i=\sum_\ell \mu_\ell^{\mathcal{U}} t_i^\ell.$$

Since the public values v, h, and u_j are identical across all authority key shares, they can be factored outside the Lagrange interpolation. Consequently,

$$\begin{aligned}\bar{K}_{i,0}&=\left[\sum_{\ell\in U}\mu_\ell^{\mathcal{U}}\big(\lambda_i^\ell+\Delta r_i^\ell-u_{\rho(i)}t_i^\ell\big)\right]_2\\&=\big[\bar{\lambda}_i+\Delta\bar{r}_i-u_{\rho(i)}\bar{t}_i\big]_2\\\bar{K}_{i,1}&=[\bar{r}_i]_2,\qquad \bar{K}_{i,2}=[\bar{t}_i]_2\end{aligned}$$

By the reconstruction property of the underlying LSSS, for every authorized set R_b,

$$\sum_{i\in R_b}\omega_i\bar{\lambda}_i=\sum_{\ell\in U}\mu_\ell^{\mathcal{U}}\sum_{i\in R_b}\omega_i\lambda_i^\ell=\sum_{\ell\in U}\mu_\ell^{\mathcal{U}}\alpha_\ell=\alpha.$$

Now consider any batch index $b\in[B]$ and any row $i\in R_b$, where $\rho(i)\in x_b$. Using bilinearity of the pairing, we obtain

$$\begin{aligned}e(C_{b,1},\bar{K}_{i,0})&=e\big([s_b]_1,[\bar{\lambda}_i+\Delta\bar{r}_i-u_{\rho(i)}\bar{t}_i]_2\big)\\&=[s_b\bar{\lambda}_i+s_b\Delta\bar{r}_i-s_bu_{\rho(i)}\bar{t}_i]_T,\end{aligned}$$

$$e(C_{b,2},\bar{K}_{i,1})=e\big([\Delta s_b]_1,[\bar{r}_i]_2\big)=[s_b\Delta\bar{r}_i]_T,$$

$$e(C_{b,\rho(i)},\bar{K}_{i,2})=e\big([u_{\rho(i)}s_b]_1,[\bar{t}_i]_2\big)=[s_bu_{\rho(i)}\bar{t}_i]_T.$$

Substituting these expressions into the decryption equation yields

$$D_{b,i}=\frac{e(C_{b,1},\bar{K}_{i,0})}{e(C_{b,2},\bar{K}_{i,1})\,e(C_{b,\rho(i)},\bar{K}_{i,2})}=[s_b\bar{\lambda}_i]_T.$$

Aggregating over all rows in the authorized set R_b, we obtain

$$D_b=\prod_{i\in R_b}D_{b,i}^{\omega_i}=\Big[s_b\sum_{i\in R_b}\omega_i\bar{\lambda}_i\Big]_T=[\alpha s_b]_T.$$

Finally, decryption recovers

$$\frac{C_{b,0}}{D_b}=\frac{m_b\cdot[\alpha s_b]_T}{[\alpha s_b]_T}=m_b.$$

Since the ciphertext digest is generated consistently during encryption, the verification step succeeds. Hence, every authorized decryptor reconstructs the original message correctly, establishing the correctness of the scheme.

Theorem 4 (Completeness). *The Construction 5.1 satisfies completeness.*

Proof. For any authority ℓ, access structure $(\mathbf{M}, \rho)$, digest d, and tag tg, the key share $\sigma_\ell = \mathsf{CompKeyShare}(\mathsf{sk}_\ell, \mathbf{M}, \rho), d, \mathsf{tg})$ is produced with the correct digest stored and with $K_{i,0}, K_{i,1}, K_{i,2}$ satisfying the defining exponent relation by construction. The auxiliary commitments $\{[y_j]_1\}_{j=2}^{k}$ allow the verifier to compute $[\lambda_i^*]_T$ correctly, so the pairing check in $\mathsf{VerifyKeyShare}$ reduces to verifying the exponent relation, which holds with probability 1.

Theorem 5 (Succinctness). *The Construction 5.1 is succinct.*

Proof. The key share σ_ℓ contains $3\ell_0$ group elements in $\mathbb{G}_2$, $(k-1)$ group elements in $\mathbb{G}_1$, and one field element in $\mathbb{Z}_p$; the aggregated key $\bar{\sigma}$ contains $3\ell_0$ group elements in $\mathbb{G}_2$ and one field element in $\mathbb{Z}_p$ (the batch digest d). Both sizes are independent of the batch size B and of the number of participating authorities $|U|$.

Theorem 6 (Static Security). *Assume that (i) the KeyGen algorithm is executed honestly, equivalently, the trusted dealer generates all key shares according to Construction 4.1; (ii) the DBDH assumption holds in the bilinear group BG; and (iii) the hash function H is modelled as a random oracle. Then Construction 4.1 satisfies static security.*
More precisely, for every admissible probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ issuing at most Q random-oracle queries, there exists a probabilistic polynomial-time DBDH distinguisher $\mathcal{B}$ such that

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{TBABE}}(\lambda) \leq 2B \cdot \mathsf{Adv}_{\mathcal{B}}^{\mathsf{DBDH}}(\lambda) + \frac{\mathrm{poly}(Q, B)}{p}.$$

Proof. The security proof is established through a standard hybrid argument. To this end, we consider a sequence of hybrid games indexed by $j \in \{0, 1, \ldots, B\}$ and parameterized by the challenge bit $\beta \in \{0, 1\}$. The interaction between the challenger and the adversary $\mathcal{A}$ is defined as follows.

- $\mathsf{Game}_0^{(\beta)}$**:** This is the real security experiment.
- $\mathsf{Game}_j^{(\beta)}$ **$(1 \leq j \leq B)$.** This game is identical to $\mathsf{Game}_{j-1}^{(\beta)}$ except that the j-th challenge ciphertext component is modified. In particular, the term $C_{j,0}$ is generated as
$$C_{j,0} = m_\beta^{(j)} \cdot Z_j,$$
where $Z_j \xleftarrow{\$} \mathbb{G}_T$ is chosen uniformly at random and independently of all other variables.
- $\mathsf{Game}_B^{(\beta)}$**.** In the final hybrid, all challenge components $\{C_{b,0}\}_{b \in [B]}$ are independently and uniformly distributed in $\mathbb{G}_T$.

The following lemma shows that any two consecutive hybrids are computationally indistinguishable under the DBDH assumption.

Lemma 3. *For every $j \in [B]$ and every challenge bit $\beta \in \{0,1\}$, there exists a PPT distinguisher $\mathcal{B}_{j,\beta}$ such that*

$$\left|\Pr[\mathsf{Game}_{j-1}^{(\beta)} = 1] - \Pr[\mathsf{Game}_{j}^{(\beta)} = 1]\right| \leq \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,\beta}}(\lambda).$$

Proof. We construct $\mathcal{B}_{j,\beta}$ that receives a DBDH instance $\mathsf{par} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [a]_1, [a]_2, [b]_1, [b]_2, [c]_1)$ together with a challenge $Z \in \mathbb{G}_T$ (either $Z = [abc]_T$ or $Z \xleftarrow{\$} \mathbb{G}_T$), and simulates the security experiment for $\mathcal{A}$.

Simulation of mpk. $\mathcal{B}_{j,\beta}$ programs the random oracle so that $H(\mathsf{tg}^*) = \tau^*$ for a fresh $\tau^* \xleftarrow{\$} \mathbb{Z}_p$. It implicitly sets

$$\alpha = ab + \tilde{\alpha}, \quad v = -b\tau^* + \tilde{v}, \quad h = b + \tilde{h}, \quad u_i = \tilde{u}_i \ (i \in [N]),$$

where $\tilde{\alpha}, \tilde{v}, \tilde{h}, \tilde{u}_i \xleftarrow{\$} \mathbb{Z}_p$. Then $v + h\tau^* = \tilde{v} + \tilde{h}\tau^*$, so the b-dependence cancels at the challenge tag tg^*. The public components are computable:

$$[\alpha]_T = e([a]_1, [b]_2) \cdot [\tilde{\alpha}]_T, \quad [v]_1 = [\tilde{v}]_1 \cdot [b]_1^{-\tau^*}, \quad [h]_1 = [\tilde{h}]_1 \cdot [b]_1, \quad [u_i]_1 = [\tilde{u}_i]_1.$$

Threshold share simulation. Fix any polynomial f of degree at most $T-1$ with

$$f(0) = 1 \qquad \text{and} \qquad f(\ell) = 0 \ \text{ for all } \ell \in C.$$

Such an f exists because the $|C| + 1 \leq T$ interpolation conditions are consistent for a degree-$(T-1)$ polynomial (when $|C| + 1 < T$ the choice is not unique, and any solution works equally well below); its coefficients $f_0 = 1, f_1, \ldots, f_{T-1}$ are known to the reduction. Independently, the reduction samples a uniformly random polynomial $\tilde{f}$ of degree at most $T-1$ subject to $\tilde{f}(0) = \tilde{\alpha}$ (i.e. it draws the non-constant coefficients $\tilde{f}_1, \ldots, \tilde{f}_{T-1} \xleftarrow{\$} \mathbb{Z}_p$). It then implicitly defines the Shamir polynomial as

$$f_\alpha(x) = ab \cdot f(x) + \tilde{f}(x), \qquad \text{hence} \qquad \alpha_\ell = f_\alpha(\ell) = ab\, f(\ell) + \tilde{f}(\ell).$$

We have $f_\alpha(0) = ab \cdot 1 + \tilde{\alpha} = \alpha$, and the non-constant coefficients of f_α are $ab\, f_j + \tilde{f}_j$. Since the $\tilde{f}_j$ are uniform and independent and $ab\, f_j$ are fixed offsets, the coefficients $(a_1, \ldots, a_{T-1})$ are uniform and independent. Therefore f_α is distributed exactly as a uniformly random degree-$(T-1)$ polynomial with constant term α, i.e. as an honest (T, L)-Shamir sharing of α.

The coefficient commitments and verification keys are computable in $\mathbb{G}_T$:

$$[a_j]_T = [\tilde{f}_j]_T \cdot e([a]_1, [b]_2)^{f_j}, \qquad \mathsf{pk}_\ell = [\alpha_\ell]_T = [\tilde{f}(\ell)]_T \cdot e([a]_1, [b]_2)^{f(\ell)} = \prod_{j=0}^{T-1} [a_j]_T^{\ell^j}.$$

For every $\ell \in C$, we have $f(\ell) = 0$. Then, $\alpha_\ell = \tilde{f}(\ell)$ is a *known* field element. The reduction hands $\mathcal{A}$ the corrupted secret shares $\{\alpha_\ell = \tilde{f}(\ell)\}_{\ell \in C}$ together with the public trapdoors. These are consistent with $\{\mathsf{pk}_\ell\}$ and with

the commitments $\{[a_j]_T\}$ by construction, and (because $|C| < T$) they leave α information-theoretically undetermined: the ab direction of f_α is carried solely by the honest evaluations $\{f(\ell)\}_{\ell \notin C}$, none of which $\mathcal{A}$ learns in the clear. The reduction returns $\big(\mathsf{pp}, \mathsf{mpk}, \{\mathsf{pk}_\ell\}_{\ell \in [L]}, \{\alpha_\ell\}_{\ell \in C}\big)$ to $\mathcal{A}$.

Key-share query simulation. Consider a query $(\ell, (\mathbf{M}, \rho), \mathcal{X}, \mathsf{tg})$ with $\mathbf{M} \in \mathbb{Z}_p^{\ell_0 \times k}$ and $\tau = H(\mathsf{tg})$. We write the (known) quantities

$$F := f(\ell), \qquad A_\ell := \tilde{f}(\ell), \qquad \Phi := \tilde{v} + \tilde{h}\tau,$$

so that $\alpha_\ell = A_\ell + abF$ and, we have $v + h\tau = \Phi + b\Delta$ with $\Delta := \tau - \tau^*$. The reduction samples the LSSS randomness $y_2, \ldots, y_k \xleftarrow{\$} \mathbb{Z}_p$ and the auxiliary randomness $\tilde{r}_i, \tilde{t}_i \xleftarrow{\$} \mathbb{Z}_p$ for every row $i \in [\ell_0]$.

(i) Case-I ($\mathsf{tg} \neq \mathsf{tg}^*$)**:** The reduction implicitly sets

$$r_i = \tilde{r}_i - \frac{M_{i,1}\, F\, a}{\Delta}, \qquad t_i = \tilde{t}_i.$$

We expand the exponent of $K_{i,0}$. First,

$$\lambda_i = M_{i,1}\alpha_\ell + \sum_{l=2}^{k} M_{i,l} y_l = \underbrace{M_{i,1} A_\ell + \sum_{l=2}^{k} M_{i,l} y_l}_{\hat{\lambda}_i \ \text{(known)}} + M_{i,1} F\, ab,$$

and, multiplying out $(v + h\tau) r_i = (\Phi + b\Delta)\big(\tilde{r}_i - \frac{M_{i,1} F a}{\Delta}\big)$,

$$\begin{aligned} (v + h\tau) r_i &= \Phi \tilde{r}_i - \frac{\Phi M_{i,1} F}{\Delta}\, a + \Delta \tilde{r}_i\, b - M_{i,1} F\, ab \\ \implies \lambda_i + (v + h\tau) r_i - u_{\rho(i)} t_i &= \underbrace{\big(\hat{\lambda}_i + \Phi \tilde{r}_i - \tilde{u}_{\rho(i)} \tilde{t}_i\big)}_{\sigma_i} + a \underbrace{\Big(-\tfrac{\Phi M_{i,1} F}{\Delta}\Big)}_{\gamma_i} + b \underbrace{\big(\Delta \tilde{r}_i\big)}_{\delta_i} \end{aligned}$$

The share components are therefore computable from the DBDH instance:

$$K_{i,0} = [\sigma_i]_2 \cdot [a]_2^{\gamma_i} \cdot [b]_2^{\delta_i}, \qquad K_{i,1} = [r_i]_2 = [\tilde{r}_i]_2 \cdot [a]_2^{-M_{i,1} F/\Delta}, \qquad K_{i,2} = [\tilde{t}_i]_2,$$

together with the auxiliary commitments $[y_l]_1 = g_1^{y_l}$, $l = 2, \ldots, k$. Because $(\tilde{r}_i, \tilde{t}_i, y_l)$ are uniform and independent, the induced (r_i, t_i, y_l) have exactly the distribution of an honest key share for the share value α_ℓ; the embedded $\tilde{f}(\ell)$ guarantees the correct marginal of α_ℓ. Thus σ_ℓ is perfectly simulated, for an arbitrary policy $(\mathbf{M}, \rho)$, whether or not it is authorised for the challenge attributes.

(ii) Case-II ($\mathsf{tg} = \mathsf{tg}^*$; **admissibility forces every x_b^* to be unauthorised for $(\mathbf{M}, \rho)$**)**:** Here $\Delta = 0$, we have $v + h\tau^* = \Phi = \tilde{v} + \tilde{h}\tau^*$ with no b-term, and the cancellation of Case 1 is unavailable for the term $M_{i,1} F\, ab$. Instead we use the unauthorised structure of the policy. Partition the rows into $R^* = \{i : \rho(i) \in$

$\bigcup_b x_b^*\}$ and its complement. Because each x_b^* is unauthorised, $\boldsymbol{e}_1 \notin \mathrm{span}\{\mathbf{M}_i : i \in R^*\}$, so there is a vector $\boldsymbol{w} = (w_1, \ldots, w_k)^\top$ with $w_1 = 1$ and $\langle \mathbf{M}_i, \boldsymbol{w}\rangle = 0$ for all $i \in R^*$. The reduction implicitly defines the LSSS randomness through $\boldsymbol{y} = \alpha_\ell \boldsymbol{w} + \tilde{\boldsymbol{y}}$ with $\tilde{\boldsymbol{y}} \xleftarrow{\$} \mathbb{Z}_p^k$ and $\tilde{y}_1 = 0$; then for rows $i \in R^*$, the share $\lambda_i = \langle \mathbf{M}_i, \boldsymbol{y}\rangle = \langle \mathbf{M}_i, \tilde{\boldsymbol{y}}\rangle$ is independent of the ab-carrying value α_ℓ and hence computable, Under that assumption the key share is reproduced without ever forming $[ab]_2$, so σ_ℓ is simulated correctly.

Challenge. $\mathcal{A}$ submits $\mathbf{m}_0 = (m_1^{(0)}, \ldots, m_B^{(0)})$ and $\mathbf{m}_1 = (m_1^{(1)}, \ldots, m_B^{(1)})$. The reduction produces the B challenge ciphertexts as follows.

- **Already-randomised ciphertexts $b < j$.** Sample fresh $s_b, z_b \xleftarrow{\$} \mathbb{Z}_p$ and set

$$C_{b,0} = m_\beta^{(b)} \cdot [z_b]_T, \quad C_{b,1} = [s_b]_1, \quad C_{b,2} = [s_b]_1^{\tilde{v}+\tilde{h}\tau^*}, \quad C_{b,k} = [s_b]_1^{\tilde{u}_k}\ (k \in x_b^*).$$

- **Embedded ciphertext $b = j$.** Implicitly set the encryption randomness $s_j = c$ and put

$$\begin{aligned} C_{j,1} &= [c]_1, & C_{j,k} &= [c]_1^{\tilde{u}_k}\ (k \in x_j^*), \\ C_{j,2} &= [c]_1^{\tilde{v}+\tilde{h}\tau^*}, & C_{j,0} &= m_\beta^{(j)} \cdot Z \cdot e([c]_1, g_2)^{\tilde{\alpha}}. \end{aligned}$$

If $Z = [abc]_T$ then $C_{j,0} = m_\beta^{(j)} \cdot [abc]_T \cdot [\tilde{\alpha}c]_T = m_\beta^{(j)} \cdot [\alpha c]_T$, i.e. an honest encryption with randomness c; this is the distribution of $\mathsf{Game}_{j-1}^{(\beta)}$. If $Z \xleftarrow{\$} \mathbb{G}_T$ then $C_{j,0}$ is uniform in $\mathbb{G}_T$ and independent of the message; this is the distribution of $\mathsf{Game}_j^{(\beta)}$.

- **Honest ciphertexts $b > j$.** Sample fresh $s_b \xleftarrow{\$} \mathbb{Z}_p$ and set

$$\begin{aligned} C_{b,0} &= m_\beta^{(b)} \cdot e([a]_1, [b]_2)^{s_b} \cdot [\tilde{\alpha} s_b]_T, & C_{b,1} &= [s_b]_1, \\ C_{b,2} &= [s_b]_1^{\tilde{v}+\tilde{h}\tau^*}, & C_{b,k} &= [s_b]_1^{\tilde{u}_k}. \end{aligned}$$

All components are computable from the instance and the reduction's own randomness.

Output. $\mathcal{B}_{j,\beta}$ forwards $\mathcal{A}$'s guess as its own DBDH decision. By the two cases above, when $Z = [abc]_T$, the view of $\mathcal{A}$ is exactly $\mathsf{Game}_{j-1}^{(\beta)}$ and when $Z \xleftarrow{\$} \mathbb{G}_T$, it is exactly $\mathsf{Game}_j^{(\beta)}$, up to the random-oracle failure event $\{\exists\, \mathsf{tg} \neq \mathsf{tg}^* : H(\mathsf{tg}) = \tau^*\}$, which by the union bound over the $O(Q+B)$ random-oracle evaluations occurs with probability at most $\mathrm{poly}(Q,B)/p$. Hence,

$$\left|\Pr[\mathsf{Game}_{j-1}^{(\beta)} = 1] - \Pr[\mathsf{Game}_j^{(\beta)} = 1]\right| \leq \mathsf{Adv}_{\mathcal{B}_{j,\beta}}^{\mathsf{DBDH}}(\lambda) + \frac{\mathrm{poly}(Q,B)}{p},$$

which is the claim.

Lemma 4. $\Pr[\mathsf{Game}_B^{(0)}(\mathcal{A}) = 1] = \Pr[\mathsf{Game}_B^{(1)}(\mathcal{A}) = 1]$.

Proof. In $\mathsf{Game}_B^{(\beta)}$ every message-carrying component has the form $C_{b,0}^* = m_\beta^{(b)} \cdot Z_b$ with $Z_b \xleftarrow{\$} \mathbb{G}_T$ independent of all other variables. The map $z \mapsto m_\beta^{(b)} \cdot z$ is a bijection of $\mathbb{G}_T$, so each $C_{b,0}^*$ is uniform in $\mathbb{G}_T$ and statistically independent of β. Every remaining component of the challenge ciphertexts (namely $C_{b,1}, C_{b,2}, \{C_{b,k}\}$) is generated from the attribute sets x_b^* and fresh encryption randomness and is therefore also independent of β. The same holds for all key shares and public keys. Consequently, the entire view of $\mathcal{A}$ is identical for $\beta = 0$ and $\beta = 1$, and the two acceptance probabilities coincide.

Putting the hybrids together. Fix an admissible $\mathcal{A}$. By Lemma 3, for each $\beta \in \{0, 1\}$,

$$\left|\Pr[\mathsf{Game}_0^{(\beta)} = 1] - \Pr[\mathsf{Game}_B^{(\beta)} = 1]\right| \leq \sum_{j=1}^{B} \left(\mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,\beta}}(\lambda) + \tfrac{\mathrm{poly}(Q,B)}{p}\right).$$

Combining this with Lemma 4 through the triangle inequality,

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{TB\text{-}ABE}}_{\mathcal{A}}(\lambda) &= \left|\Pr[\mathsf{Game}_0^{(0)} = 1] - \Pr[\mathsf{Game}_0^{(1)} = 1]\right| \\
&\leq \left|\Pr[\mathsf{Game}_0^{(0)} = 1] - \Pr[\mathsf{Game}_B^{(0)} = 1]\right| + \underbrace{\left|\Pr[\mathsf{Game}_B^{(0)} = 1] - \Pr[\mathsf{Game}_B^{(1)} = 1]\right|}_{=0 \text{ (Lemma 4)}} \\
&\quad + \left|\Pr[\mathsf{Game}_B^{(1)} = 1] - \Pr[\mathsf{Game}_0^{(1)} = 1]\right| \\
&\leq \sum_{j=1}^{B} \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,0}}(\lambda) + \sum_{j=1}^{B} \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}_{j,1}}(\lambda) + \frac{\mathrm{poly}(Q,B)}{p}.
\end{aligned}$$

Letting $\mathcal{B}$ be the distinguisher among $\{\mathcal{B}_{j,\beta}\}_{j\in[B],\beta\in\{0,1\}}$ with the largest advantage gives

$$\mathsf{Adv}^{\mathsf{TB\text{-}ABE}}_{\mathcal{A}}(\lambda) \leq 2B \cdot \mathsf{Adv}^{\mathsf{DBDH}}_{\mathcal{B}}(\lambda) + \frac{\mathrm{poly}(Q,B)}{p},$$

as claimed. Whenever DBDH holds and p is super-polynomial in λ, the right-hand side is negligible, so the construction is adaptively secure against static corruptions.

Acknowledgments. This work is supported in part by the Anusandhan National Research Foundation (ANRF), Department of Science and Technology (DST), Government of India, under File No. EEQ/2023/000164 and the Information Security Education and Awareness (ISEA) Project Phase-III initiatives of the Ministry of Electronics and Information Technology (MeitY) under Grant No. F.No. L-14017/1/2022-HRD.

References

1. Agarwal, A., Das, S., Gilkalaye, B.P., Rindal, P., Shoup, V.: Btx: Simple and efficient batch threshold encryption. Cryptology ePrint Archive (2026)

2. Agarwal, A., Fernando, R., Pinkas, B.: Efficiently-thresholdizable batched identity based encryption, with applications. In: Annual International Cryptology Conference. pp. 69–100. Springer (2025)
3. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in bft networks. Cryptology ePrint Archive (2022)
4. Beimel, A.: Secure schemes for secret sharing and key distribution. Technion-Israel Institute of technology, Faculty of computer science (1996)
5. Bethencourt, J., Sahai, A., Waters, B.: Ciphertext-policy attribute-based encryption. In: 2007 IEEE symposium on security and privacy (SP'07). pp. 321–334. IEEE (2007)
6. Boldyreva, A.: Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme. In: International Workshop on Public Key Cryptography. pp. 31–46. Springer (2002)
7. Boneh, D., Boyen, X.: Secure identity based encryption without random oracles. In: Annual International Cryptology Conference. pp. 443–459. Springer (2004)
8. Boneh, D., Franklin, M.: Identity-based encryption from the weil pairing. In: Annual international cryptology conference. pp. 213–229. Springer (2001)
9. Boneh, D., Laufer, E., Tas, E.N.: Threshold batch identity-based encryption without epochs. Cryptology ePrint Archive, Paper 2025/1254 (2025), https://eprint.iacr.org/2025/1254
10. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the weil pairing. In: ASIACRYPT. pp. 514–532. Springer (2001)
11. Boneh, D., Nema, R., Roy, A., Tas, E.N.: Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive (2026)
12. Chatterjee, S., Hankerson, D., Menezes, A.: On the efficiency and security of pairing-based protocols in the type 1 and type 4 settings. In: International Workshop on the Arithmetic of Finite Fields. pp. 114–134. Springer (2010)
13. Chatterjee, S., Hankerson, D., Menezes, A.: On the efficiency and security of pairing-based protocols in the type 1 and type 4 settings. In: In International Workshop on the Arithmetic of Finite Fields. pp. 114–134. Springer (2010)
14. Desmedt, Y.: Threshold cryptosystems. In: international workshop on the theory and application of cryptographic techniques. pp. 1–14. Springer (1992)
15. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th annual symposium on foundations of computer science (sfcs 1987). pp. 427–438. IEEE (1987)
16. Gong, J., Waters, B., Wee, H., Wu, D.J.: Threshold batched identity-based encryption from pairings in the plain model. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 548–578. Springer (2026)
17. Goyal, V., Pandey, O., Sahai, A., Waters, B.: Attribute-based encryption for fine-grained access control of encrypted data. In: Proceedings of the 13th ACM conference on Computer and communications security. pp. 89–98 (2006), https://doi.org/10.1145/1180405.1180418
18. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: International conference on the theory and application of cryptology and information security. pp. 177–194. Springer (2010)
19. Lewko, A., Okamoto, T., Sahai, A., Takashima, K., Waters, B.: Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 62–91. Springer (2010)

20. Lewko, A., Waters, B.: New techniques for dual system encryption and fully secure hibe with short ciphertexts. In: theory of cryptography conference. pp. 455–479. Springer (2010)
21. Lewko, A., Waters, B.: Decentralizing attribute-based encryption. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 568–588. Springer (2011)
22. Oh, D., Kim, J., Oh, H.: Lightbeat: Scalable epochless batched threshold encryption via hierarchical identity-based puncturing. IEEE Access (2026)
23. Rouselakis, Y., Waters, B.: Practical constructions and new proof methods for large universe attribute-based encryption. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. pp. 463–474. ACM (2013)
24. Sahai, A., Waters, B.: Fuzzy identity-based encryption. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 457–473. Springer (2005), https://doi.org/10.1007/11426639_27
25. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979)
26. Shamir, A.: Identity-based cryptosystems and signature schemes. In: Workshop on the theory and application of cryptographic techniques. pp. 47–53. Springer (1984)
27. Waters, B.: Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In: Advances in Cryptology – CRYPTO 2009. LNCS, vol. 5677, pp. 619–636. Springer (2009)
28. Waters, B.: Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In: International workshop on public key cryptography. pp. 53–70. Springer (2011)
29. Xiong, S., Zhang, Y., Chen, J.: Efficient batched ibe from lattices. Cryptology ePrint Archive (2025)